

基于多模型融合的 NIPT 时点优化与胎儿异常判定研究

摘 要

本文旨在解决无创产前检测 (NIPT) 中的时点选择优化与胎儿异常判定问题, 通过构建一系列数学模型, 为不同特征的孕妇群体提供科学、个性化的 NIPT 策略。研究不仅关注 Y 染色体浓度与关键生理指标的关系, 还深入探讨了检测误差的影响, 并为女胎异常判定提供了可靠的数据驱动方法。

针对问题一, 为探究胎儿 Y 染色体浓度与孕妇孕周及 BMI 的相关特性, 本文首先通过对比多种全局回归模型, 论证了分层控制变量法的必要性。为寻求最优分层方案, 本文比较了 K-Means 聚类、等百分位数法等五种分组策略, 并以加权 R^2 为指标确定了最优方案。在此基础上, 通过建立分层线性回归模型并结合 Fisher 组合检验进行显著性分析, 最终得出结论: 孕周与 Y 染色体浓度呈显著正相关, 而 BMI 呈显著负相关, 模型具有高度统计显著性 (p 值均小于 10^{-7})。

针对问题二, 为给不同 BMI 区间的男胎孕妇确定最佳 NIPT 时点以最小化潜在风险, 本文构建了一个三阶段优化模型。首先, 采用 Cox 比例风险模型, 结合“BMI 三次方”与“初始孕周”作为协变量 (C-index 高达 0.852), 精确预测每位孕妇 Y 染色体浓度达标的个体最佳时点。其次, 拟定平衡检测延后和过早的风险函数, 求得使组内总风险最小的组最佳时点 NIPT。考虑到其受到分组情况的影响, 因而建立动态规划模型, 通过以参数调和风险函数和组平均时点的总成本, 来统筹优化 BMI 分组以及最佳时点。最后, 通过构建综合误差分数 E 来量化检测误差, 对比全样本与高质量“稳健数据集”的分析结果, 证实检测误差会导致推荐时点平均延后约 0.43 周, 从而量化了其对决策的影响。

针对问题三, 为在更复杂场景下提供可靠的 NIPT 时点建议, 本文将模型拓展至多维协变量。首先, 构建了包含身高、年龄等多因素的多元 Cox 比例风险模型, 以更全面地评估风险。随后, 对问题二的动态规划算法进行改进, 加入了“达标率阈值”和“分组样本数阈值”的硬性约束, 以确保分组方案的临床可行性。通过再次对比全样本与稳健数据集的优化结果, 进一步验证了检测误差对模型的影响, 发现在高质量数据下, 模型能够进行更精细的分组 (7 组 vs. 5 组), 且平均最佳时点可提前 0.64 周。

针对问题四, 为解决女胎染色体异常判定的难题, 特别是在样本类别严重不平衡 (异常样本仅占 10.95%) 的挑战下, 本文提出了一套鲁棒的分类建模方案。首先, 采用随机上采样、随机下采样及 SMOTE 过采样技术处理数据不平衡问题。然后, 构建了基于“One-vs-Rest”策略的 L1 正则化逻辑回归模型, 对 21、18、13 号染色体异常进行分类预测。通过分层 Kfold 交叉验证和 ROC 分析, 结果表明, 经过随机上采样处理后的模型在召回率等关键指标上表现最佳, 显著提升了对少数类异常样本的识别能力, 证明了该方法的有效性和可靠性。

关键词: NIPT 时点优化 Cox 比例风险模型 动态规划 分层回归分析 类别不平衡

一、 问题重述

1.1 问题背景

无创产前检测 (NIPT) 通过分析孕妇外周血中的胎儿 DNA 片段, 已成为一种安全、准确的产前筛查技术。然而, 对于男胎而言, 其 Y 染色体浓度在母体血液中的水平直接影响 NIPT 的准确性。过低的 Y 染色体浓度可能导致检测结果不确定, 增加重复检测的需要, 从而带来额外的成本和心理负担。因此, 深入研究胎儿 Y 染色体浓度与孕妇生理指标的关系, 并据此优化 NIPT 的检测时机, 对于提升检测效用和降低相关风险具有重要意义。

1.2 题目信息

- 胎儿 Y 染色体浓度水平受到多种因素影响, 并随孕周动态变化。
- 检测误差是 NIPT 过程中不可避免的, 需要对其进行评估和控制。
- 除了孕周, 孕妇的 BMI、身高、体重、年龄等生理指标也可能对胎儿 Y 染色体浓度产生影响。
- NIPT 策略需在保证达标率的前提下, 尽早检测以降低孕妇的潜在风险, 这是一个多目标优化的过程。
- 附件提供了匿名化的胎儿 Y 染色体浓度检测数据, 用于模型建立和验证。

1.3 待求解问题

问题一: 探究孕妇 BMI 和初始孕周等因素对 Y 染色体浓度达标时间的影响, 并对模型显著性进行检验。

问题二: 根据孕妇 BMI 对数据进行合理分组, 并为每组确定最佳 NIPT 时点, 以综合最小化组内风险和最佳时点总和。

问题三: 设计误差分析方法, 通过筛选高质量样本验证模型稳健性, 并量化检测误差对最佳 NIPT 时点的影响, 以提升方案的可靠性。

问题四: 综合利用 X 染色体及各常染色体的 Z 值、GC 含量、测序读数指标和孕妇 BMI 等多维度特征, 对女胎的 21 号、18 号和 13 号染色体异常状态建立准确的染色体异常分类判定模型。

二、模型假设与符号说明

2.1 模型假设

- 假设胎儿 Y 染色体浓度的检测误差主要来源于测序数据质量（比对比例、重复比例等指标）
- 假设无极端身体状况或罕见疾病对染色体浓度的异常影响
- 排除多胎妊娠等特殊妊娠情况的影响
- 假设样本数据中不存在测量仪器完全失效或人为操作重大失误的情况

2.2 符号说明

表 1 符号说明

符号	意义
N	样本总数
β_i	Cox 模型中协变量的系数，值反映第 i 个 BMI 分组对风险率的影响程度
n_k	第 k 个分组的样本数
T	NIPT 最佳时点，单位为周
T_{ideal}	胎儿 Y 染色体浓度最早达标时间
T_k	第 k 个分组的最佳 NIPT 时点
$h_0(t)$	基准风险函数，表示在所有协变量都为 0 时的风险
λ	动态规划成本函数中的权重参数
E	综合误差分数
C_k	第 k 个分组，由一组样本构成
C-index	一致性指数，表示模型对达标时间排序的准确性
Y_{ij}	第 i 位孕妇第 j 次检测的 Y 染色体浓度
GA_{ij}	第 i 位孕妇第 j 次检测的孕周数
BMI_{ij}	第 i 位孕妇第 j 次检测时的 BMI
u_{0i}	线性混合效应模型中的随机截距，代表第 i 位孕妇的基线差异
u_{1i}	线性混合效应模型中的随机斜率，代表第 i 位孕妇孕周效应的个体差异
ε_{ij}	线性混合效应模型中的残差项
X	包含孕周、BMI、身高、年龄等生理指标的协变量矩阵

三、 问题分析

3.1 问题总分析

针对无创产前检测 (NIPT) 在临床应用中面临的时点选择优化和胎儿异常判定两大核心挑战, 本文旨在构建一个全面、系统的数学模型框架。该框架将通过数据驱动的方法, 为不同生理特征的孕妇提供个性化、科学化的 NIPT 策略, 并探究影响检测结果的关键因素。

3.2 具体问题分析

3.2.1 问题一的分析

问题一要求探究胎儿游离 DNA 中 Y 染色体浓度与孕妇的孕周和 BMI 这两项生理指标之间的相关性。该问题本质上是一个数据驱动的相关性分析和影响因素量化问题, 为解决这一挑战, 本文将分析过程细化为三个关键步骤。其一, 通过对比全局回归模型与分层模型, 验证了分层控制变量法的必要性, 以更准确地反映不同孕妇群体的内在规律。其二, 为寻求最优分层方案, 本文比较了 K-Means 聚类、等百分位数法等五种分组策略, 并以加权指标作为评估标准, 最终确定出最能反映数据内在结构的分层方案。最后, 基于已确定的最优分层, 本文建立了分层线性回归模型, 并引入双重显著性检验和 Fisher 组合检验, 从统计学角度证明了所揭示的相关性并非随机现象, 为后续问题中的模型构建提供了严谨的数学基础。

3.2.2 问题二的分析

问题二要求在男胎孕妇群体中, 对 BMI 进行合理分组, 并为每组确定最佳的 NIPT 时点, 以达到组内风险最小化和最佳时点总和最小化的综合目。该问题本质上是一个多目标优化问题, 即需要在尽可能降低检测风险 (例如, 尽早识别高危胎儿) 与避免过早检测导致准确性下降之间找到平衡点。同时, 题目明确指出需要考量检测误差对分析结果的影响。为解决上述问题, 本文将分析过程细化为以下三个关键步骤: 首先, 运用 Cox 比例风险模型来预测每位孕妇的胎儿 Y 染色体浓度最早达到 4% 这一阈值的具体时点。其次, 建立动态规划模型, 以 BMI 指标作为分组依据, 对预测出的个体最佳时点进行优化分组。此步骤的目标是构建一个最优的分组方案, 使得在满足预设约束的前提下, 实现总成本函数的最小化, 即同时优化组内风险和最佳时点均值。最后, 通过筛选出高质量样本来创建稳健数据集, 并与全样本数据集的结果进行对比, 量化并分析检测误差对 NIPT 最佳时点选择的影响, 以评估模型的稳健性和可靠性。

3.2.3 问题三的分析

问题三作为问题二的延伸, 要求在综合考虑更多生理指标 (如身高、体重、年龄等)、检测误差以及 Y 染色体浓度达标比例等多种因素的前提下, 为孕妇提供更为可靠的分组策略和最佳 NIPT 时点建议。与问题二相比, 该问题增加了模型的复杂度和约束条件。因此, 本文采取的分析思路主要包括以下方面: 其一, 将问题二中使用的 Cox 比例风险模型扩展为多元 Cox 比例风险模型。该模型将纳入更广泛的生理指标作为协变量, 以更全面地评估这些因素对 Y 染色体浓度达标时间的影响。其二, 沿用问题二中的动态规划方法, 但在此基础上引入额外的约束条件以确保所得模型结果在临床应用中的实用性和可靠性。其三, 延续问题二的思路, 通过比较在高质量样本数据集和全样本数据集上运行的优化结果, 进一步量化和分析检测误差在考虑更复杂生理指标和约束条件下的影响,

从而验证模型的泛化能力和鲁棒性。

3.2.4 问题四的分析

问题四要求综合分析女胎孕妇的 21 号、18 号和 13 号染色体非整倍体数据，并结合 X 染色体 Z 值、GC 含量等多种生理及检测指标，为女胎染色体异常状况建立一个可靠的判定方法。同时该数据集中正常样本与异常样本数量存在严重不平衡，故而本文进行以下分析：首先应解决样本类别不均衡问题，于是对原数据集做随机采样处理，使得不同类别的数据均衡；进一步需构建多分类逻辑回归模型，通过 L1 正则化有效解决了模型的过拟合问题，并通过对特征的筛选，简化模型的同时确保了模型具有良好的泛化能力和解释性；最后，引入 Kfloat 交叉检验进行模型的稳健性检验，可进一步证实模型的强泛化能力。

四、 数据预处理

为确保数据质量，并使其适用于后续的量化分析与建模，我们对原始数据进行了整合、清洗与变量转换。

4.1 数据清洗与变量处理

我们利用 Python 对原始数据中的缺失值和异常值进行了处理，并统一了数据格式，具体步骤如下：

4.1.1 缺失值填补

针对“末次月经”变量中的缺失值，我们基于数据中已有的“检测日期”和“检测孕周”信息，通过计算方法对其进行估算与填补。

4.1.2 数据校验与校正

我们根据孕妇的身高和体重信息重新计算了 BMI 值，并与原始数据中的 BMI 值进行了对比。对于 188 条记录中计算值与原始值不一致的情况，我们采用重新计算的 BMI 值进行了校正，以保证数据准确性。

4.1.3 日期格式统一

原始数据中“末次月经”和“检测日期”的记录格式不统一（分别为“2023/2/1”和“20230429”）。为便于数据处理，我们统一将所有日期格式转换为“YYYYMMDD”的格式，以确保数据的一致性。

原始数据中孕周的 w 大小写出现了不一致的情况，我们将其统一转为小写的 w。

4.1.4 异常值筛选

根据 GC 含量（G-C 碱基对占总碱基对的比例）的正常范围（40%~60%），我们考虑到约 40%的数据集中在 39.5%-40%之间，因此将正常范围扩展至 39.5%~60.5%。我们保留了在此区间内的数据，并剔除了 39 个超出此范围的异常值，以确保数据质量。

4.2 变量转换与数值编码

为将分类变量转换为适用于模型分析的数值型变量，我们使用 Stata 软件对部分关键变量进行了编码，具体方法如下：

4.2.1 “IVF 妊娠”变量转换

我们首先确保该变量为字符串类型，随后将其转换为新的数值型变量“IVF_numeric”，并进行如下编码：

- 将“自然受孕”赋值为 1。
- 将“IVF（试管婴儿）”赋值为 2。
- 将“IUI（人工授精）”赋值为 3。

4.2.2 “怀孕次数”变量转换

我们首先确保该变量为字符串类型，随后将其转换为新的数值型变量“pregnancytime_numeric”，并进行如下编码：

- 将怀孕次数值小于等于 3 次的记录保留原始值。
- 将怀孕次数值显示为“≥3”的记录统一赋值为 4。

4.2.3 “胎儿是否健康”变量转换

我们首先确保该变量为字符串类型，随后将其转换为新的数值型变量“babyhealth_numeric”，并进行如下编码：

- 将“是”赋值为 1。
- 将“否”赋值为 0。

五、 问题一的模型建立与求解

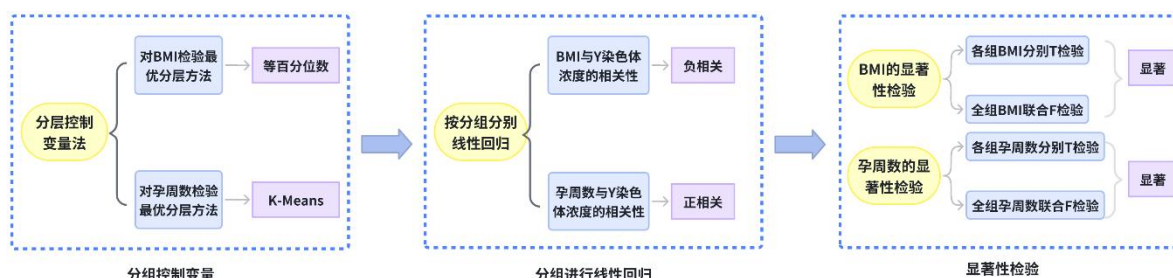


图 2 问题一主体思路

5.2 问题聚焦

本研究旨在探究胎儿 Y 染色体浓度与孕妇的孕周数（GA）和身体质量指数之间的相关性及其特性，并构建一个能够解释这些关系的数学模型，同时对模型的显著性进行检验。通过该分析，我们将为后续研究男胎 Y 染色体浓度达标时间（即浓度达到或超过 4% 的最早时间）提供坚实的理论基础和数据支持。

5.3 模型的建立与解决

5.3.1 分析策略确立与最优分组方法遴选

在分析初期，我们尝试采用全局回归模型（如 Spearman 相关性分析、广义可加模型等）直接对全体样本进行拟合，但结果表明，单一模型无法有效捕捉数据内部的异质性，拟合优度欠佳。为此，我们决定采用更为精细化的分层控制变量法。该策略的核心思想是：在研究某一变量（如孕周）的影响时，先将另一相关变量（如 BMI）进行分组，在每个组内单独分析，从而剥离变量间的耦合效应，使目标关系更清晰地显现。

此策略的成功与否高度依赖于分组方法的科学性。为确保分组的有效性，我们对五种潜在的分组方法进行了综合评估：等距分层、K-Means 聚类分组、等百分位数分层、基于效应变化趋势的分层以及临床经验分组。我们以加权决定系数（Weighted R^2 ）作为

核心评价指标，该指标能量化评估不同分组方案下，模型整体的解释能力。经过全面的比较分析，我们最终确定了最优的分组策略：

分析孕周效应时：采用 K-Means 聚类法对孕妇 BMI 进行分组，效果最为理想。

分析 BMI 效应时：采用等百分位数法对孕周进行分组，表现最佳。

5.3.2 分层回归建模与核心相关关系揭示

在确定了最优分组方案的基础上，我们进入建模求解阶段。针对每个通过 K-means 聚类（按 BMI 分组）或等百分位数法（按孕周分组）划分出的数据子集，我们分别建立了线性回归模型，以定量描述变量间的关系。通过对所有子模型的结果进行归纳分析，我们揭示出两条明确且一致的核心规律，如下图所示：

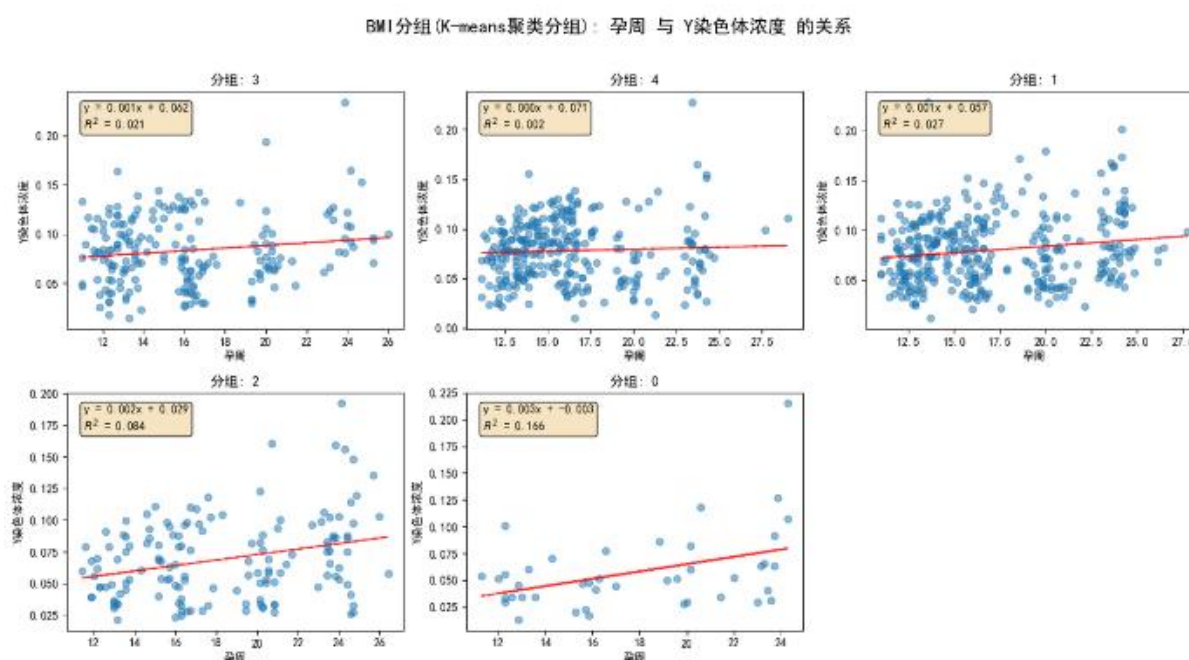


图 3 对 BMI 分组与 Y 染色体浓度的线性回归结果

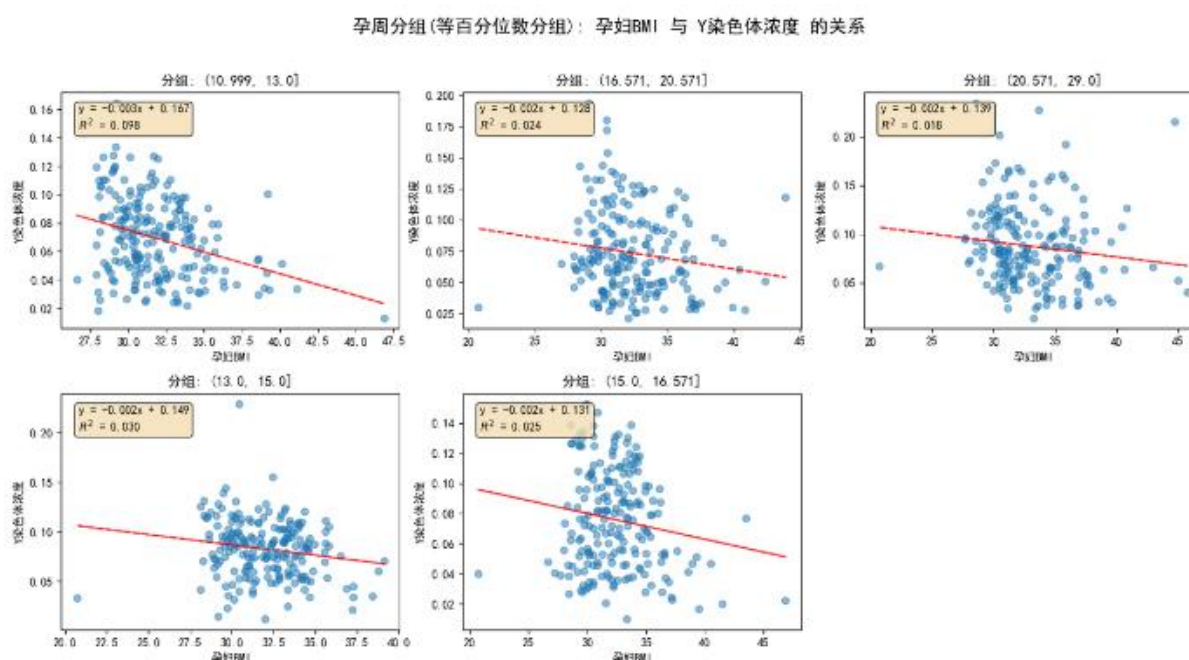


图4 孕周分组与 Y 染色体浓度的线性回归结果

如图所示，孕周与 Y 染色体浓度呈正相关：在控制了孕妇 BMI 的影响后，胎儿的 Y 染色体浓度会随着孕周的增加而系统性地上升；BMI 与 Y 染色体浓度呈负相关：在控制了孕周的影响后，孕妇的 BMI 越高，其胎儿的 Y 染色体浓度则呈现出越低的趋势，这与孕妇的 BMI 越大则胎儿游离 DNA 比例越低¹文献结论一致。

5.3.3 模型显著性检验与结论验证

为确保上述结论的统计学严谨性，我们设计了双重显著性检验流程。首先，我们对每个子组内的线性回归模型进行组内显著性检验。我们采用以下公式来判断模型的显著效应：

$$\text{显著效应} = \frac{\text{显著组数}}{\text{总组数}} \times 100\% \quad (1)$$

结果显示，在两种最优分层策略下， p 值显著 ($p < 0.05$) 的子模型占比均超过 80%，这表明我们在绝大多数局部区间内发现的相关关系是稳定且可靠的。

随后，为了评估整个分层分析框架的总体有效性，我们采用 Fisher 组合检验，对所有子组的 p 值进行合并分析。该方法能够给出一个评估全局显著性的整体 p 值。检验结果如下：

表 2 对 p 值进行评估

	显著效应比例	整体回归 p 值	Fisher 组合检验 p 值
按 BMI 分组分析孕周效应	80.00%	5.04E-07	2.27E-08
按孕周分组分析 BMI 效应	80.00%	1.94E-08	4.44E-08

由表可知，这两个 p 值均达到了极高的显著性水平，有力地证明了我们所构建的分层关系模型是稳健有效的，其揭示的孕周正相关性与 BMI 负相关性并非随机波动，而是具有坚实统计学基础的客观规律。

六、问题二的模型建立与求解

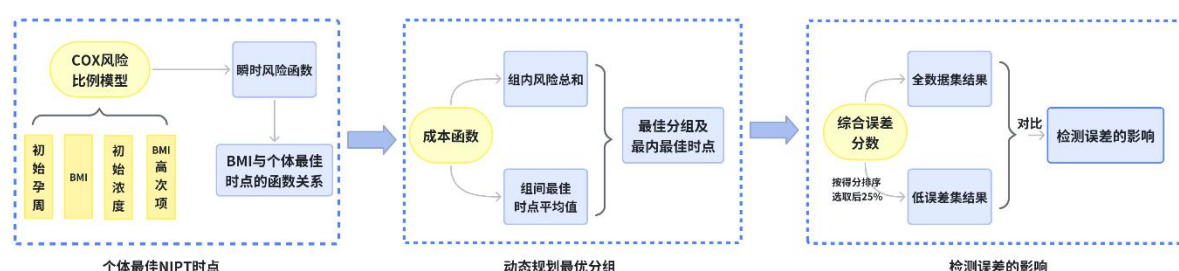


图5 问题二主题思路

6.2 问题聚焦

本问题旨在通过分析男胎孕妇的 BMI 与其胎儿 Y 染色体浓度最早达标时间（即浓度达到或超过 4% 的最早时间）之间的关系，合理划分 BMI 区间，并为各区间确定一个最佳的 NIPT 时点，以期最小化孕妇潜在的检测风险。同时，需要分析检测误差对结果

¹ 薛莹,丁洁,贺权泽,等.孕妇年龄、孕周及体重指数对孕妇外周血中胎儿游离 DNA 比例的影响[J].中国产前诊断杂志(电子版),2017,9(03):5-10.DOI:10.13470/j.cnki.cjpd.2017.03.002.

的影响。

针对此问题，我们首先将模型建立分解为三个关键步骤：

1. **个体最佳 NIPT 时点的确定：**基于实际观测数据，对每位男胎孕妇个体，利用生存分析方法（如半参数 Cox 比例风险模型）预测其 Y 染色体浓度达到 4% 的最早孕周数。
2. **BMI 与个体现有最佳时点的动态优化：**在确定了个体最佳 NIPT 时点后，我们将孕妇按其 BMI 值进行合理分组。利用动态优化算法，结合 BMI 和个体最佳 NIPT 时点信息，将孕妇划分为具有相似特征的群体。
3. **检测误差对结果的影响：**根据划分的群体，我们引入综合误差分数（E）来量化测序数据质量。通过比较稳健数据集（E 值处于后 25% 的高质量样本）和全样本数据集的分析结果，我们旨在深入探讨和验证检测误差如何导致最佳 NIPT 时点预测的偏差，并量化其对模型预测性能和临床决策的潜在影响。

这种分步建模的策略，首先关注个体层面的精细化分析，再进行群体层面的聚合与风险评估，旨在更准确地刻画 BMI 与 NIPT 时点间的复杂关系。

6.3 模型的建立与求解

6.3.1 个体最佳 NIPT 时点确定

首先，我们将事件定义为：

事件 A: Y 染色体浓度 $\geq 4\%$

对于每位男胎孕妇个体，我们基于其相关的临床数据，特别是孕周数和 Y 染色体浓度变化信息，利用 Cox 比例风险模型来估计个体最佳 NIPT 时点。

Cox 模型能够分析协变量（如基础风险函数，此处可视为随孕周变化的基线风险）对事件发生概率的影响。通过将孕周数作为时间变量，并对个体数据进行建模，我们可以估计出每位孕妇 Y 染色体浓度达到 4% 的生存时间，即个体最佳 NIPT 时点。虽然 Cox 模型本身侧重于风险率的相对变化，但通过其累积风险函数，我们可以估算出事件发生的概率，并据此推断出个体首次达到 4% 浓度阈值的预测时间点。

Cox 比例风险模型是广泛应用于生物医学领域的概率预测模型²，它是一种在生存分析中广泛使用的统计模型，用于研究一个或多个协变量（如年龄、性别、治疗方式）如何影响某个事件（如死亡、疾病复发）的发生率或风险。

其能够有效分析协变量对事件发生风险的影响，且无需预设达标时间的具体分布。该模型的数学形式为：

$$h(t|X) = h_0(t) \cdot \exp(\beta_1 \cdot X_1 + \beta_2 \cdot X_2 + \dots + \beta_p \cdot X_p) \quad (2)$$

其中，

- $h(t|X)$ 代表在时间 t 时，给定协变量分组下的瞬时风险率。
- $h_0(t)$ 是基础风险函数，表示当所有协变量为零时的风险率。
- β_i 是模型系数，其值反映了第 i 个协变量分组对风险率的影响程度。 $\exp(\beta_i)$ 为风险比（Hazard Ratio, HR），HR 大于 1 表示该分组的风险更高（即达标时间更早）。

为了为后续分析提供基础，我们首先对每位男胎孕妇个体进行数据分析，确定其 Y 染色体浓度首次达到或超过 4% 时的孕周数。该孕周数即为该孕妇个体的“达标时间”，记为 T_i 。

²王坤,梁惠雅,熊建军.基于 Cox 和 BP 神经网络组合模型的中国水务企业风险预警研究[J].财会研究,2024,(11):60-72.

为了构建一个预测能力最强的模型，我们尝试了多种协变量组合，并利用一致性指数（C-index）来评估模型的性能。C-index 是生存分析中常用的评估指标，C-index 越高，模型的预测能力越强。

通过比较不同协变量组合的 C-index，我们确定了最优模型为“BMI 三次方”与“初始孕周”的组合。该模型的 C-index 达到了 0.852，表现出很强的时间预测能力。模型的具体表达式为：

$$h(t|X) = h_0(t) \cdot ((-0.0001 \cdot BMI_{centered}^3) + (-0.0001 \cdot \text{初始孕周})) \quad (3)$$

其中， $BMI_{centered}^3$ 代表个体 BMI 减去平均 BMI 后的三次方。我们选择 BMI 的三次方而非原始值或二次方，是因为该非线性变换在模型中表现出最高的一致性，能够更好地捕捉 BMI 与达标时间之间的复杂关系。

模型中的基础风险函数 $h_0(t)$ 是一个基于数据的非参数函数，它代表了当所有协变量值都为零时的达标瞬时风险。该函数没有简单的数学表达式，通常以阶梯函数（查找表）的形式存在。与 $h_0(t)$ 相关联的是基准生存函数，表达如下：

$$S_0(t) = \exp(-H_0(t)) = \exp\left(-\int_0^t h_0(u) du\right) \quad (4)$$

它表示在所有协变量为零的情况下，Y 染色体浓度在时间 t 之前都未达到 4% 的概率。

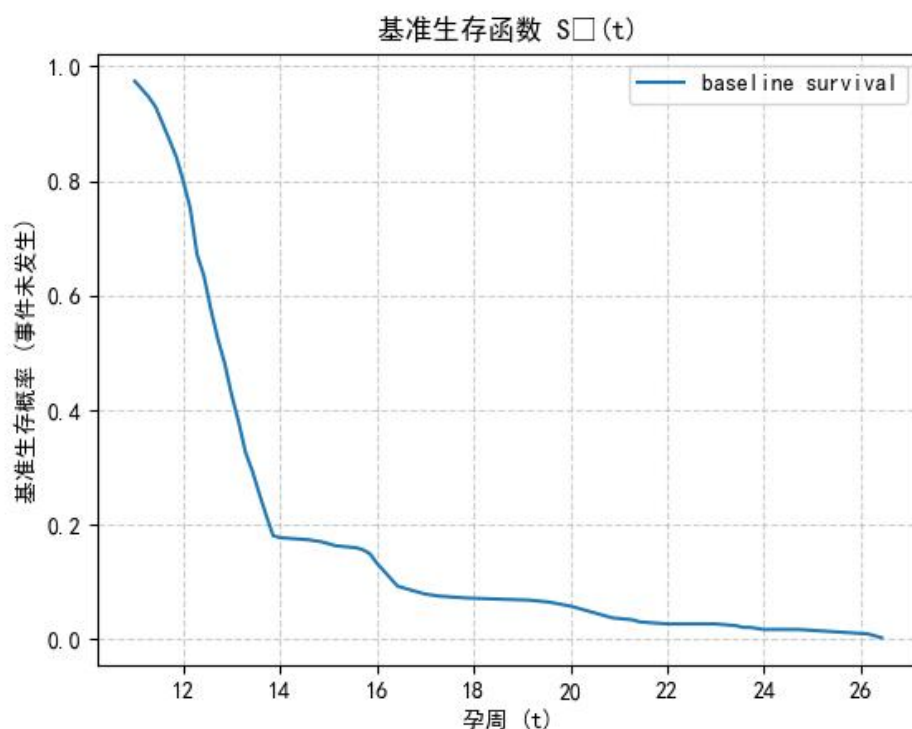


图 6 基准生存函数曲线

通过对我们数据集的分析，基准生存函数曲线显示在孕 11 周后，其值开始迅速下降，这意味着 Y 染色体浓度达到 4% 的事件开始频繁发生。在孕周 16 周左右，生存概率趋近于零，表明绝大多数孕妇的 Y 染色体浓度此时都已经达到或超过了 4%。

综上所述，我们构建的 Cox 比例风险模型可以根据每位孕妇的 BMI 和初始孕周，结合基准生存函数，精确预测其 Y 染色体浓度何时能达到 4% 这一阈值，从而为确定个体最佳 NIPT 时点提供了有力的量化依据。

6.3.2 BMI 分组与个体时点的优化分析

在确定了个体最佳 NIPT 时点（记为 T_i ）后，我们结合孕妇的 BMI 值，对样本进行优化分析。为了在降低孕妇潜在风险的同时，为不同 BMI 群体的孕妇提供最优的 NIPT 时点，我们提出了一个综合考虑组内风险和时点均值的优化分析模型：我们的目标是找到一个最优的分组方案，使得组内达标时间 T_{ideal} 的离散度（即风险）最小，同时也使各组最佳 NIPT 时点 T 的均值最小化。

我们采用动态规划方法对 BMI 数据进行优化。该方法能够确保找到全局最优解，避免陷入局部最优。此时，问题转化为一个成本最小化问题，成本函数由两部分组成：

$$Cost = \sum_{i=1}^k Risk(Group_i) + \lambda \cdot \sum_{i=1}^k T_{opt}(Group_i) \quad (5)$$

其中， $Risk(Group_i)$ 代表第 i 个 BMI 组内的达标时间离散度（我们采用平方误差作为衡量指标）， $T_{opt}(Group_i)$ 为该组的最佳 NIPT 时点（即组内 T_{ideal} 的均值）， k 为分组数量，而 λ 是用于平衡组内风险最小化和最佳时点均值最小化的权重参数。

1. **组内风险总和**：这部分旨在最小化每个分组内个体 NIPT 最佳时点 (T_{ideal}) 的离散程度。它的计算公式是：

$$\sum_{i=1}^k Risk(Group_i) = \sum_{k=1}^K A \cdot \sum_{i \in C_k} (T_i - T_k)^2 \quad (6)$$

其中，

$$A = \begin{cases} 0.83 & t > T_i \\ 0.27, & t < T_i \\ 0 & t = T_i \end{cases}$$

K 是分组数量， C_k 代表第 k 个分组， T_i 是分组内第 i 个样本的 T_{ideal} ， T_k 是第 k 个分组的最佳时点。

2. **最佳时点均值**：这部分旨在惩罚较大的 NIPT 最佳时点，鼓励模型选择更早的检测时点。它的计算公式是：

$$\lambda \cdot \sum_{i=1}^k T_{opt}(Group_i) = \frac{\lambda}{k} \sum_{k=1}^K T_k \quad (7)$$

其中， λ 是权重参数，用于平衡两部分目标； T_k 是第 k 个分组的最佳时点。

为了确定最优的 λ 值，我们利用了帕累托前沿分析。通过对 λ 进行敏感性分析，我们绘制了“总风险”与“总时点均值”的权衡曲线。在这个曲线上，每一个点都代表一个在特定 λ 下的最优分组方案，这些点组成的曲线便是帕累托前沿。通过分析曲线的拐点和结合实际的临床需求，我们选择了 $\lambda = 0.0695$ 作为最优权重参数。该值在有效控制总风险的同时，也避免了过度追求时点提前而导致的分组不合理。

最终，我们使用 $\lambda = 0.0695$ 的参数，通过动态规划算法对全体男胎孕妇数据进行优化，得到了 11 个最优的 BMI 分组，并为每个组确定了最佳 NIPT 时点。具体结果如下表所示：

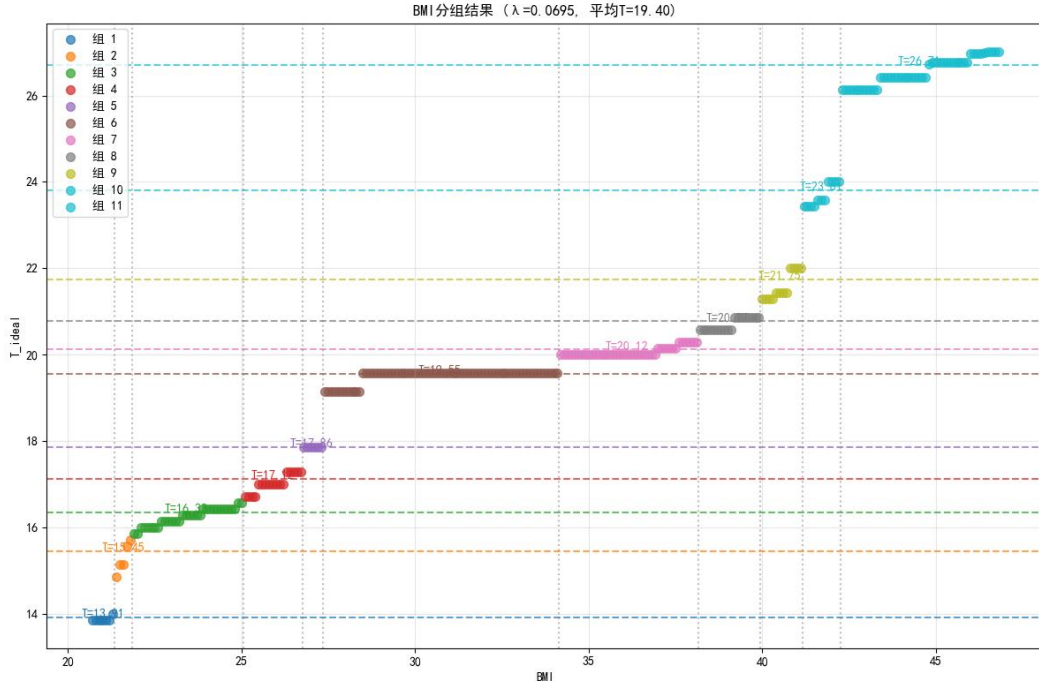


图 7 全样本动态规划分组情况

从结果可以看出，随着 BMI 的增加，最佳的 NIPT 时点也呈递增趋势，这与我们之前建立的 Cox 模型的结论相符。通过这种动态规划，我们能够为每一位孕妇提供一个基于其 BMI 的定制化、科学化的 NIPT 时点建议，从而最大限度地降低因检测时机不当而带来的潜在风险。

6.3.3 检测误差对结果的影响

为了评估检测误差对最佳 NIPT 时点的影响，我们引入了衡量测序数据质量的指标 E （综合误差分数）。

$$E = (1 - \text{比对比例}) + \text{重复比例} + (1 - \text{唯一比对读段数归一化}) + (1 - \text{GC 含量}) \quad (8)$$

该指标综合考虑了唯一比对读段数、比对比例、重复读段比例和 GC 含量等多项参数，能够量化数据的稳健性。E 值越低，代表数据质量越高，测序误差越小。我们假定，高质量的样本能够更真实地反映达标时间，从而为模型提供更准确的训练数据。

我们一方面从数据集中筛选出 E 值处于后 25% 的高质量样本，并将其定义为稳健数据集。另一方面，我们利用该稳健数据集重复了 6.2.2 节中的动态规划优化分析。通过重新进行参数调优，我们得到了针对高质量样本的最佳 λ 值，并据此获得了新的分组方案和最佳 NIPT 时点。

以下是使用最佳 λ 值 (0.0428) 对稳健数据集进行分组的最终结果：

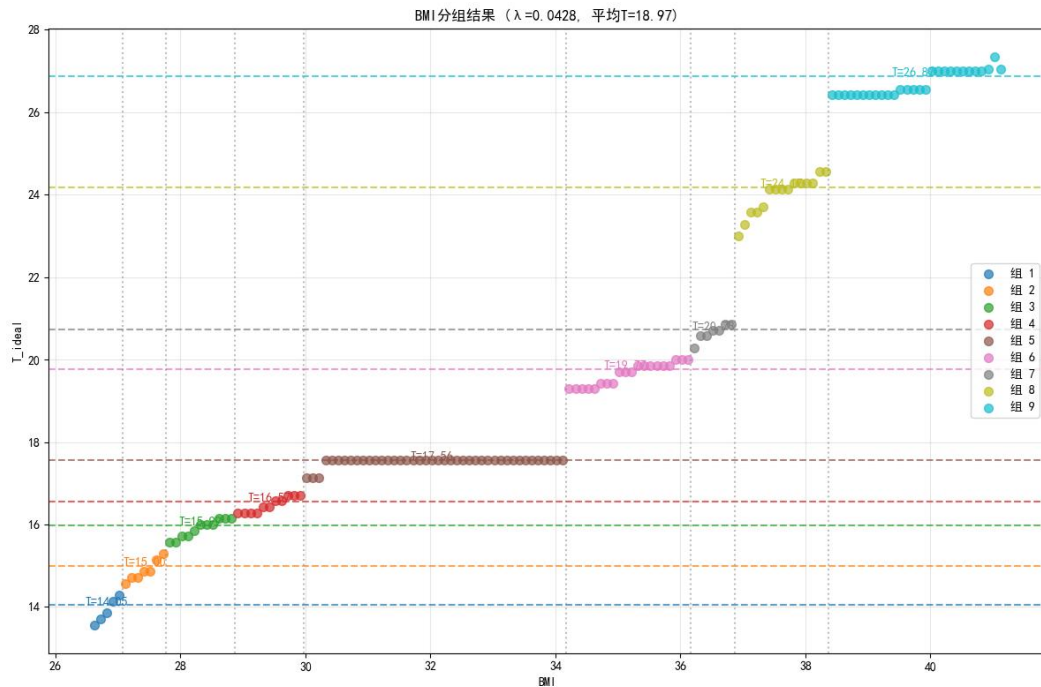


图 8 稳健性样本动态规划分组情况

经过动态规划优化，稳健数据集的最佳 λ 值、对应总风险、时点均值、平均时点与之前的全数据集有了显著差异，如下表所示：

表 4 协变量为 BMI 时动态规划优化前后最佳 λ 值

	推荐 λ 值	对应总风险	对应总 T 和	对应平均 T	对应组数
稳健数据集	0.0428	3.75	170.71	18.97	9
全数据集	0.0695	4.79	213.38	19.4	11

1. 总风险更低：稳健数据集的总风险为 3.75，低于所有样本的总风险 4.79。这表明高质量的测序数据能够有效降低模型的不确定性，使得模型预测更为可靠。
2. 最佳时点更早：稳健数据集的平均最佳 NIPT 时点为 18.97 周，早于所有样本的平均时点 19.40 周。这一结果有力地验证了我们的假设：检测误差会使得模型预测的最佳 NIPT 时点延后。这与我们基于 Cox 模型系数的定性分析结论一致，即误差大的样本通常伴随着 Y 染色体浓度偏低，导致模型倾向于预测更晚的达标时间。

七、问题三的模型建立与求解

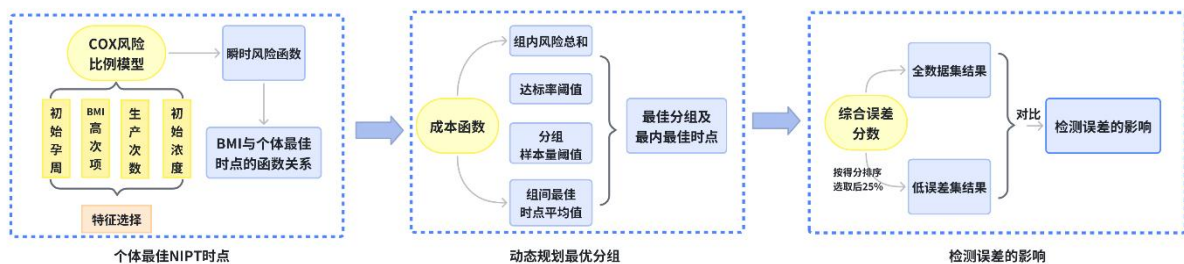


图 9 问题三主体思路

7.2 问题聚焦

本问题旨在通过综合分析多种生理指标、检测误差与 Y 染色体浓度达标比例，为男胎孕妇提供更具普适性和鲁棒性的最佳 NIPT 时点推荐方案，以最小化潜在的检测风险。

针对此问题，我们首先将模型建立分解为三个关键步骤：

1. **多元风险因素的建模：**我们拓展了单一的 BMI 分析，构建了包含初始孕周、BMI、身高、初始浓度等在内的多元 Cox 比例风险模型。为筛选出对达标时间影响最显著的变量，我们综合运用了 p 值、系数绝对值、风险比及置信区间等多种指标进行特征选择。
2. **BMI 分组与时点优化：**在确定了多元模型下的个体最佳达标时间后，我们沿用动态规划优化算法。不同的是，我们在此步骤中融入了达标率阈值和分组样本量阈值的硬性约束，以确保每个推荐时点的临床有效性和可靠性。
3. **检测误差对结果的影响分析：**为深入探究检测误差的影响，我们通过筛选出综合误差分数 E 较低的高质量样本，构建了“稳健数据集”。通过对比稳健数据集与全样本数据集在分组、最佳时点和风险总和等方面的差异，我们量化并验证了检测误差对模型预测结果的负面影响。

7.3 模型的建立与求解

7.3.1 多元 Cox 模型的建立与求解

为了更全面地评估多因素对 Y 染色体浓度达标时间的影响，我们对问题二中单一的 BMI 模型进行了扩展，引入了初始孕周、身高、初始浓度和综合误差分数 E 等协变量来建立多元 Cox 比例风险模型。

在模型建立过程中，我们并未依赖单一的特征选择方法，而是综合运用了 p 值、系数绝对值、风险比以及 95% 置信区间等多种指标。一个变量只要在任一筛选标准下表现出统计学或临床上的显著性，即被纳入最终模型，这种方法确保了模型能够捕捉到不同角度的关键影响因素，提升了模型的稳健性和解释力。

最终，我们筛选出的模型公式及其参数如下，其 C-index 达到了 0.853，表明模型具有良好的预测性能：

$$h(t|X) = h_0(t) \cdot \exp(\beta_1 \log_{\text{初始浓度} \cdot \text{生产次数}} + \beta_2 BMI_{\text{centered}}^3 + \beta_3 \text{初始孕周}) \quad (9)$$

其中 β_i 为各协变量的系数，经过计算， $\beta_1=0.0025$ ， $\beta_2=-0.0004$ ， $\beta_3=-0.4883$ 。

7.3.2 动态规划分组算法的改进与求解检测误差对结果的影响分析

在获得多元模型下的个体最佳达标时间(T_{ideal})后，我们沿用了问题二中成熟的动态规划优化算法进行分组。为了更好地满足实际临床应用需求，我们在原有的总成本函数（见公式(4)，组内风险总和与加权最佳时点总和的和）基础上，加入了两个硬性约束：

$$\textcircled{1} R_{MP} = \frac{\text{组内到达达标时间孕妇个数}}{\text{组内孕妇总数}} \geq 80\%， \text{ 以确保方案的临床有效性。}$$

其中， R_{MP} 为最小达标率（MIN_PASS_RATE）。

$$\textcircled{2} S_{MG} \geq 5， \text{ 以避免因分组过细而导致的过拟合和结果不稳定性。}$$

其中， S_{MG} 为最小分组样本量（MIN_GROUP_SIZE）。

通过对权重参数 λ 进行敏感性分析，我们最终确定了最优 λ 值。在全样本数据集上，最终分组结果如下：

表 7 全样本集分组结果

组别	BMI 区间	样本数	最佳时点 T	达标率
1	[20.70, 21.30]	7	13.91	85.70%
2	[21.40, 28.00]	67	17.31	80.60%
3	[28.10, 44.10]	161	22.22	81.40%
4	[44.20, 46.30]	22	26.79	81.80%
5	[46.40, 46.80]	5	27.01	80.00%

7.3.3 检测误差分析

为了量化检测误差对模型预测结果的影响，我们创建了一个稳健数据集。该数据集同样通过筛选综合误差分数 E 处于后 25% 的高质量样本得到，剔除了因测序质量问题可能导致数据偏差的样本。

我们在这里也使用和问题二相同的动态规划优化算法和约束条件，在稳健数据集上重复了分组和优化过程。通过对比全样本数据集与稳健数据集的分析结果，我们发现：

表 8 加入多个协变量时动态规划优化前后最佳 λ 值

	推荐 λ 值	对应总风险	对应平均 T	对应组数
稳健数据集	0.5878	52.13	20.18	7
全数据集	7.017	497.44	21.45	5

1. 分组数量差异：全样本数据被划分为 5 组，而稳健数据集被划分为 7 组。这表明在剔除误差较大的样本后，数据的内在结构变得更加精细，模型可以进行更细粒度的划分。
2. 最佳时点差异：稳健数据集的平均最佳时点为 20.81 周，而全样本数据集的平均最佳时点为 21.45 周。这一偏移量量化了检测误差对推荐时点的影响，证明了高质量数据对于早期、准确预测的重要性。

八、问题四的模型建立与求解

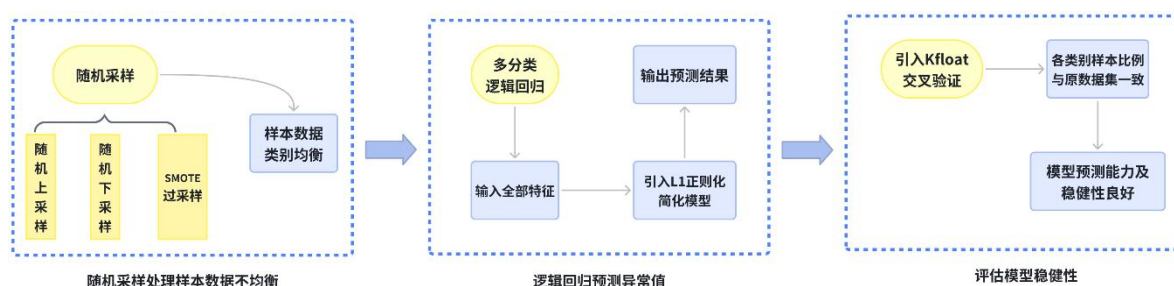


图 10 问题四主体思路

8.2 问题聚焦

本问题旨在综合分析女胎孕妇的 21 号、18 号和 13 号染色体非整倍体数据，并结合 X 染色体 Z 值、GC 含量等多种生理及检测指标，为女胎异常的判定建立一个可靠的判

断方法。由于数据集中正常样本与异常样本数量存在严重不平衡，本文将特别关注如何构建一个既能有效利用数据，又能避免过拟合，并能对异常情况进行高精度判定的模型。

针对此问题，我们将模型建立分解为三个关键步骤：

1. **处理样本类别不平衡问题：**考虑到正常女胎样本数量远多于异常女胎样本，直接在原始数据上训练模型容易导致模型偏向多数类。为解决这一问题，我们运用随机上采样、随机下采样和 SMOTE 过采样三种方法，分别构建平衡的样本数据集，为后续的建模分析提供可靠的数据基础。
2. **构建异常风险预测模型：**在处理好数据不平衡问题后，我们以各类染色体异常为判别目标，将问题转化为一个多分类问题，并选择二分类逻辑回归作为核心预测模型。通过训练独立的二分类器，我们可以计算并输出每种异常类型的预测概率，从而为临床诊断提供量化的风险评估。
3. **模型性能评估与优化：**为确保模型的泛化能力和预测准确性，我们首先引入 Lasso 正则化来对模型参数进行约束，以筛选出对异常判断最为关键的特征，并有效避免过拟合。随后，我们通过 Kfold 交叉验证对模型进行严格检验，并使用 ROC 曲线（Receiver Operating Characteristic）和 AUC 值（Area Under the Curve, ROC 曲线下的面积）等指标对模型的预测效果进行全面评估，以验证模型的稳健性和可靠性。

8.3 模型建立与检验

8.3.1 处理样本类别不平衡问题

在处理女胎异常判定的二分类问题时，我们注意到原始数据存在严重的类别不平衡。数据显示，非整倍体数量为 0（即正常样本）的样本有 504 个，而非整倍体数量不为 0（即异常样本）的样本仅有 62 个，少数类样本比例仅为 10.95%。这种不平衡性可能导致模型训练时偏向多数类，从而对异常样本的识别能力较差。为了解决这一问题，我们采取了三种不同的采样策略来构建平衡数据集：

- (1) 随机上采样：该方法通过简单地复制少数类样本，使其数量与多数类相同。它的优点是不会丢失任何信息，且实现简单，但缺点是可能因为重复样本而导致模型过拟合。经上采样后，数据集大小变为 1008，其中两类样本各占 504 个。
- (2) 随机下采样：该方法随机删除多数类样本，使其数量与少数类相同。这能减少计算量并加快训练速度，但可能因删除了重要信息而影响模型性能。经下采样后，数据集大小变为 124，其中两类样本各占 62 个。
- (3) SMOTE 过采样：该方法通过插值而非简单复制来创建合成的少数类样本，从而增加样本的多样性，有效避免了过拟合问题。经 SMOTE 过采样后，数据集大小变为 1008，其中两类样本各占 504 个。

8.3.2 构建异常风险预测模型

考虑到女胎的异常可能表现为 21 号、18 号或 13 号染色体的非整倍体，这是一个多分类问题。我们选择采用逻辑回归模型，并结合“OneVsRest”策略来处理多分类任务。该方法通过为每个类别训练一个独立的二分类器来预测其异常概率。

对于每个类别 k ，我们建立一个二分类逻辑回归模型，其核心是 Sigmoid 函数，用于将线性模型的输出映射到 0 到 1 之间的概率值。其数学表达式为：

$$P(y=1|x) = \frac{1}{1 + e^{-(\beta_0 + \sum_{j=1}^M \beta_j x_j)}} \quad (10)$$

其中， $P(y=1|x)$ 表示样本 x 属于类别 1 的概率， β_0 为截距项， β_j 为第 j 个特征的

权重，而 M 为特征数量。

为了防止过拟合并实现特征选择，我们引入了 L1 正则化，将惩罚项添加到损失函数中。正则化后的损失函数 J_k 可以表示为：

$$J_k(w^k, b^k) = -\frac{1}{N} \sum_{i=1}^N [y_i^k \log(p_i^k) + (1 - y_i^k) \log(1 - p_i^k)] + \lambda \sum_{j=1}^M |w_j^k| \quad (11)$$

其中， N 是样本数量， y_i^k 是真实标签（对于当前类别 k 为 1，其他为 0）， p_i^k 是模型预测的属于类别 k 的概率， w_j^k 是权重向量，而 λ 是正则化强度参数。通过最小化该损失函数，我们能够同时优化模型的预测精度并简化模型，从而提高其泛化能力。

根据以上的模型，我们对四个样本集（df1 代表原始数据集，df2 代表随机上采样的数据集，df3 代表随机下采样的数据集，df4 代表 SMOTE 过采样的数据集）进行回归，得到结果如下表：

表 9 二分类逻辑回归结果

数据集	k	precision	recall	f1-score	support
df1	0	0.92	1	0.96	504
	1	0.79	0.22	0.34	50
	2	1	0.42	0.59	12
df2	0	0.77	0.84	0.8	504
	1	0.75	0.71	0.73	407
	2	0.97	0.79	0.88	97
df3	0	0.78	0.82	0.8	62
	1	0.69	0.74	0.71	50
	2	1	0.42	0.59	12
df4	0	0.84	0.84	0.84	504
	1	0.79	0.83	0.81	475
	2	0.71	0.17	0.28	29

通过随机上采样（df2）处理后，模型性能得到显著提升。尽管对正常样本的精确率略有下降，但其对异常样本（类别 1 和类别 2）的召回率分别大幅提高至 0.71 和 0.79。这表明，上采样策略有效平衡了模型对各类别的识别能力，使其在保证一定查准率的前提下，显著增强了对异常胎儿的检测能力。类似地，SMOTE 过采样（df4）也在一定程度上改善了对类别 1 的识别性能，但对样本量极少的类别 2 改善有限。

综合来看，通过采用采样方法平衡数据集，我们成功地解决了原始数据中存在的类别不平衡问题，并显著提升了模型对女胎染色体异常的综合判定能力，使得模型在临床应用中更具鲁棒性和实用价值。

8.3.3 模型性能评估与优化

我们采用 Kfold 交叉验证 方法来评估模型的稳健性。由于数据集存在类别不平衡，我们选择了分层 Kfold 交叉验证，以确保在每次训练和测试中，各类别的样本比例与原始数据集保持一致，从而避免了某些折叠中完全没有少数类样本的情况。

为了全面评估模型的分类性能，我们绘制了 ROC 曲线，并计算了 AUC 值。AUC

值越大，表明模型的预测能力越强。根据模型在不同数据集上的表现，结果如下图所示：

交叉检验训练结果桑基图



图 11 交叉训练结果示意图

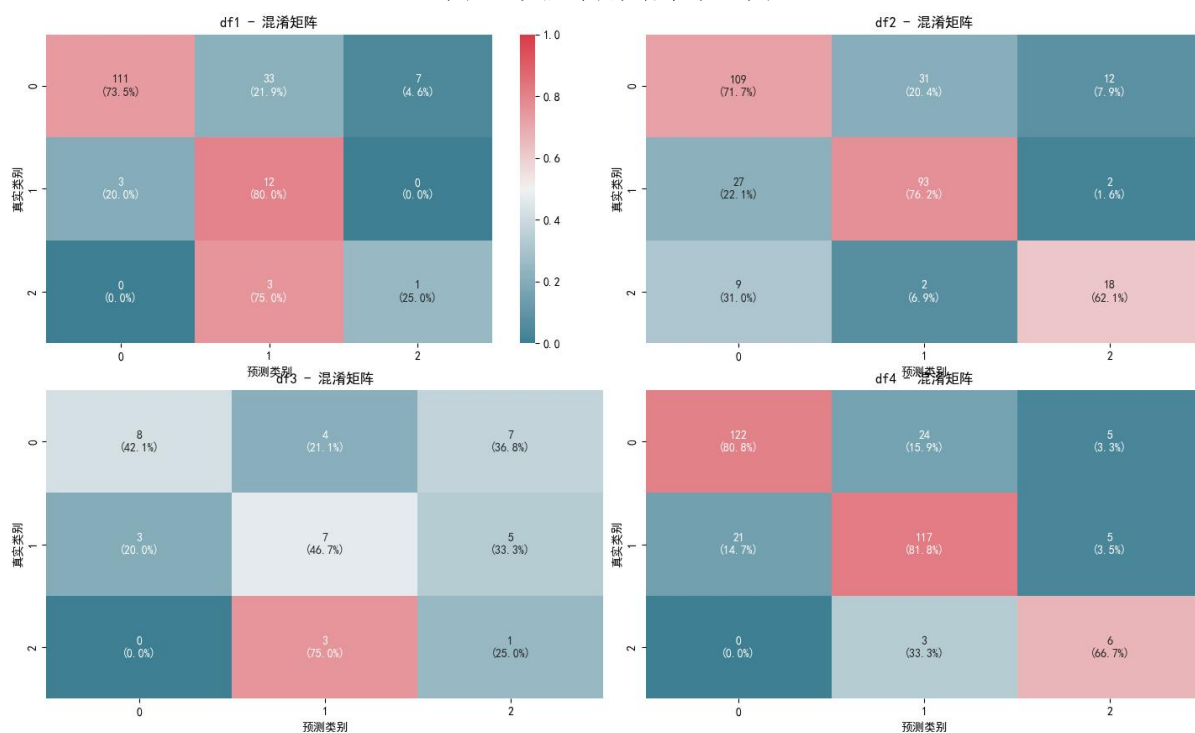


图 12 混淆矩阵

通过混淆矩阵图我们可以看出 df1 和 df3 的检测表现不如另外两个理想，其原因在于：df1 是类型失衡的原始数据集，df3 是欠采样的数据集，总样本量很少。

这些结果表明，通过平衡样本数据集，模型的整体预测性能得到了显著提升。同时，Lasso 正则化的应用有效解决了模型的过拟合问题，并通过对特征的筛选，确保了模型具有良好的泛化能力和解释性。

九、模型的分析与检验

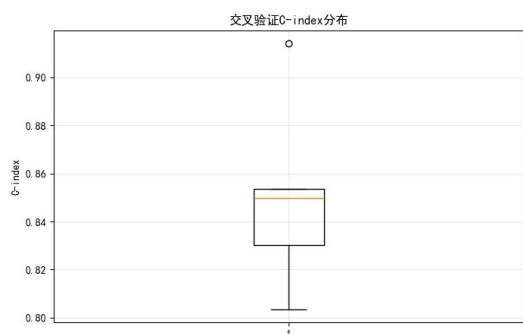
对于半参数 cox 模型，对输入的数据集训练集和测试集，通过评估模型在未见数据

上的表现来检测模型的综合性能。在 Fload 交叉检验的数据集上，模型 C-index 稳定为 0.850 (± 0.036)，说明了模型不仅良好的预测能力而且泛化能力较强。

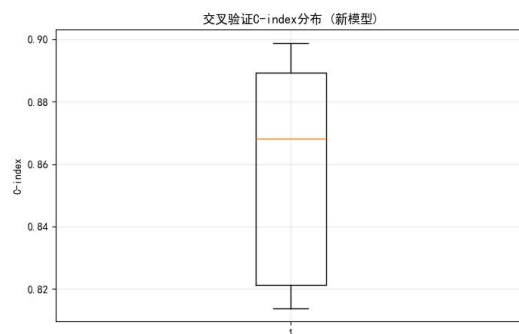
针对第三问融入新增生产次数相关变量的 cox 模型，除交叉检验外还按照生产次数对数据集分组检测 (C-index = 0.870, 0.837 以及 1.000)，皆取得了稳定的结果。

针对半参数 cox 模型的鲁棒性检测，对输入的数据集划分训练集和测试集并交叉检验，从而评估模型在未见数据上的表现来检测模型的鲁棒性。在 Kload 交叉验证的数据集上，模型 C-index 为 0.850 (± 0.036)，说明了模型不仅良好的预测能力而且预测稳定。

对于第三问新增与生产次数相关变量的 Cox 模型，除交叉检验外还按照生产次数对数据集分组检测 (C-index = 0.870, 0.837 以及 1.000)，皆取得了稳定的结果。



(a)问题二中 Cox 模型的检验



(b)问题三中 Cox 模型的检验

图 13 Cox 模型的交叉检验

十、 模型评价、改进与推广

10.1 模型的优点

1. 问题二、问题三中的 Cox 比例风险模型能够有效处理删失数据，充分利用所有可用信息；
2. 问题二、问题三中所使用的动态规划方法能够找到全局最优解，保证分组的科学性；
3. 问题四中，L1 正则化能够自动进行特征选择，提高模型可解释性；
4. 问题四中，OneVsRest 策略下的多分类逻辑回归能够提供每个样本属于各个类别的概率估计。

10.2 模型的缺点

1. 问题二、三中的动态规划计算复杂度较高，当分组数量较多时计算量较大。

10.3 模型的改进与推广

1. 应用于其他领域：模型中解决类别不平衡和多目标优化的思想和方法，可以推广应用于其他类似的医学诊断或预测问题，例如其他遗传病筛查或疾病风险评估。
2. 持续数据验证与模型迭代：持续收集新的临床数据对模型进行验证和校准，特别是在不同地区和人群中，以确保模型的持续有效性和泛化能力。

十一、 参考文献

- [1] 薛莹,丁洁,贺权泽,等.孕妇年龄、孕周及体重指数对孕妇外周血中胎儿游离 DNA 比例的影响[J].中国产前诊断杂志(电子版),2017,9(03):5-10.DOI:10.13470/j.cnki.cjpd.2017.03.002.
- [2] 王坤,梁惠雅,熊建军.基于 Cox 和 BP 神经网络组合模型的中国水务企业风险预警研究[J].财会研究,2024,(11):60-72.
- [3] 刘秋芳.无创产前基因检测在双胎妊娠非整倍体检测中的应用及效果研究[D].华南理工大学,2020.DOI:10.27151/d.cnki.ghnlu.2020.001822.
- [4] 薛莹,丁洁,贺权泽,等.孕妇年龄、孕周及体重指数对孕妇外周血中胎儿游离 DNA 比例的影响[J].中国产前诊断杂志(电子版),2017,9(03):5-10.DOI:10.13470/j.cnki.cjpd.2017.03.002.
- [5] Zhao Q, HuoJiaBieKe J, Du S. The influence of fetal gender and maternal characteristics on fetal cell-free DNA in maternal plasma. J Gynecol Obstet Hum Reprod. 2019 Oct;48(8):653-656. doi: 10.1016/j.jogoh.2019.07.001. Epub 2019 Jul 2. PMID: 31276844.
- [6] Zhou Y, Zhu Z, Gao Y, et al. Effects of maternal and fetal characteristics on cell-free fetal DNA fraction in maternal plasma[J]. Reproductive Sciences, 2015, 22(11): 1429-1435.

十二、 附录

附录 1

支撑材料的文件列表

文件名	类型	简介
数据预处理.py	Python 代码文件	数据预处理
Q1 探索 1) .py	Python 代码文件	问题 1 探索尝试
Q1 探索 2) .py	Python 代码文件	问题 1 探索尝试
Q1 分组 try.py	Python 代码文件	问题 1 最终做法
Q2 求解函数.py	Python 代码文件	问题 2 半参数 cox 模型求解
Q2 动态约束.py	Python 代码文件	问题2 动态规划分组以及最佳时点
Q3 特征选择.py	Python 代码文件	问题 3 筛选特征重要性
Q3 求解 2.py	Python 代码文件	问题 3 新变量优化半参数 cox
Q3 约束动态规划.py	Python 代码文件	问题 3 新约束下求解
Q4_logist.py	Python 代码文件	问题 4 逻辑回归判定

附录 2

数据预处理.py

统一日期格式, 填补缺失, 处理孕周数, 剔除 GC 异常值, 校验 BMI 数值, 生产次数和年龄

```
001 import pandas as pd
002 import numpy as np
003 import matplotlib.pyplot as plt
004 from datetime import datetime, timedelta
005
006 plt.rcParams['font.sans-serif'].insert(0, 'SimHei')
007 plt.rcParams['axes.unicode_minus'] = False
008 import matplotlib
009 matplotlib.use('TkAgg')
010
011 def preprocess_nipt_data(df):
012     '''统一日期格式, 填补缺失, 处理孕周数, 校验 BMI 数值, 剔除 GC 异常值'''
013
014     original_count = len(df)
015
016     # 统一检测日期格式为 YYYYMMDD
017     def format_test_date(date_val):
018         if pd.isna(date_val):
019             return np.nan
020
021         if isinstance(date_val, str) and len(date_val) == 8 and date_val.isdigit():
022             return date_val
023
024         try:
025             # 处理 datetime 对象
026             if isinstance(date_val, (pd.Timestamp, datetime)):
```

```

027         return date_val.strftime("%Y%m%d")
028
029     date_str = str(date_val).strip()
030
031     if ' ' in date_str:
032         date_str = date_str.split()[0]
033
034     for fmt in ("%Y/%m/%d", "%Y-%m-%d", "%Y/%m/%d", "%m/%d/%Y", "%d/%m/%Y"):
035         try:
036             date_obj = datetime.strptime(date_str, fmt)
037             return date_obj.strftime("%Y%m%d")
038         except:
039             continue
040
041     date_obj = pd.to_datetime(date_str, errors='coerce')
042     if not pd.isna(date_obj):
043         return date_obj.strftime("%Y%m%d")
044
045     return np.nan
046 except:
047     return np.nan
048
049 df['检测日期'] = df['检测日期'].apply(format_test_date)
050
051 def format_last_period(date_val):
052     if pd.isna(date_val):
053         return np.nan
054     if isinstance(date_val, (pd.Timestamp, datetime)):
055         return date_val.strftime("%Y%m%d")
056     try:
057         date_obj = datetime.strptime(str(date_val).split()[0], "%Y-%m-%d")
058         return date_obj.strftime("%Y%m%d")
059     except:
060         return np.nan
061
062 # 统一末次月经格式
063 df['末次月经'] = df['末次月经'].apply(format_last_period)
064
065 # 处理检测孕周列
066 def parse_gestational_week(week_str):
067     if pd.isna(week_str):
068         return np.nan, np.nan
069
070     try:
071         parts = str(week_str).lower().replace('+', '').split('w')
072         weeks = int(parts[0])
073         days = int(parts[1]) if len(parts) > 1 and parts[1].isdigit() else 0
074
075         # 计算孕周小数 (保留3位小数)
076         gestational_week = weeks + days / 7
077         # 计算孕天
078         gestational_days = weeks * 7 + days
079
080         return round(gestational_week, 3), gestational_days
081     except:
082         return np.nan, np.nan
083
084 parsed = df['检测孕周'].apply(lambda x: parse_gestational_week(x))
085 df['孕周'] = [x[0] for x in parsed]
086 df['孕天'] = [x[1] for x in parsed]

```

```

087
088 # 填充末次月经缺失值
089 def fill_missing_last_period(row):
090     if pd.isna(row['末次月经']) and not pd.isna(row['检测日期']) and not pd.isna(row['孕天']):
091         try:
092             # 检测日期已经是 YYYYMMDD 格式, 直接解析
093             test_date = datetime.strptime(row['检测日期'], "%Y%m%d")
094
095             # 计算末次月经 = 检测日期 - 孕天
096             last_period = test_date - timedelta(days=int(row['孕天']))
097             return last_period.strftime("%Y%m%d")
098         except:
099             return np.nan
100     return row['末次月经']
101
102 df['末次月经'] = df.apply(fill_missing_last_period, axis=1)
103
104 # 检查并修正 BMI 值
105 df['original_bmi'] = df['孕妇 BMI']
106
107 def check_and_fix_bmi(row):
108     if pd.isna(row['体重']) or pd.isna(row['身高']):
109         return row['孕妇 BMI']
110
111     try:
112         # 计算 BMI (体重 kg / (身高 m)^2)
113         height_m = row['身高'] / 100
114         calculated_bmi = row['体重'] / (height_m ** 2)
115         calculated_bmi = round(calculated_bmi, 3) # 保留 3 位小数
116
117         if pd.isna(row['孕妇 BMI']) or abs(row['孕妇 BMI'] - calculated_bmi) > 0.001:
118             return calculated_bmi
119         return row['孕妇 BMI']
120     except:
121         return row['孕妇 BMI']
122
123 df['孕妇 BMI'] = df.apply(check_and_fix_bmi, axis=1)
124
125 # 统计 BMI 修改情况
126 modified_bmi_mask = (
127     (~df['original_bmi'].isna() & (df['original_bmi'] != df['孕妇 BMI'])) |
128     (df['original_bmi'].isna() & ~df['孕妇 BMI'].isna())
129 )
130 modified_bmi_count = modified_bmi_mask.sum()
131
132 df.drop('original_bmi', axis=1, inplace=True)
133
134 # 删除 GC 含量不在正常范围的数据 (39.5% < GC 含量 < 60%)
135 if 'GC 含量' in df.columns:
136     # 转换为百分比 (0.399399 → 39.9399%)
137     gc_percent = df['GC 含量'] * 100
138     # 筛选条件: 保留 GC 含量在 39.5%到 60%之间的数据
139     df = df[(gc_percent >= 39.5) & (gc_percent <= 60)]
140
141 # 记录处理后的数据量
142 processed_count = len(df)
143 deleted_count = original_count - processed_count
144
145 print(f"数据预处理完成:")
146 print(f"- 原始数据量: {original_count}")

```

```

147     print(f"- 删除数据量: {deleted_count}")
148     print(f"- 保留数据量: {processed_count}")
149     print(f"- BMI 修改行数: {modified_bmi_count}")
150
151     return df
152
153
154 def fix_production_count_with_stats(df):
155     """处理生产次数不唯一的情况"""
156     if "孕妇代码" not in df.columns or "生产次数" not in df.columns:
157         raise ValueError("数据框中缺少'孕妇代码'或'生产次数'列")
158
159     df_fixed = df.copy()
160
161     # 记录原始生产次数用于比较
162     original_counts = df_fixed["生产次数"].copy()
163
164     # 按孕妇数据分组, 将生产次数替换为该组的最大值
165     df_fixed["生产次数"] = df_fixed.groupby("孕妇代码")["生产次数"].transform('max')
166
167     # 比较原始值和修改后的值, 找出被修改的记录
168     changed_mask = (original_counts != df_fixed["生产次数"])
169     changed_count = changed_mask.sum()
170
171     # 输出修改统计信息
172     if changed_count > 0:
173         # 获取被修改的孕妇列表
174         changed_mothers = df_fixed.loc[changed_mask, "孕妇代码"].unique()
175         print(f"共修改了 {changed_count} 条记录, 涉及 {len(changed_mothers)} 位孕妇")
176
177         # 显示修改示例 (前 5 条)
178         sample_changes = df_fixed[changed_mask].head(5)
179         for idx, row in sample_changes.iterrows():
180             print(f"孕妇 {row['孕妇代码']}: {original_counts[idx]} -> {row['生产次数']}")
181     else:
182         print("没有记录被修改, 所有孕妇的生产次数都是唯一的")
183
184     return df_fixed, changed_count
185
186
187 def fix_age_sequence_detailed(df):
188     """处理孕妇年龄序列异常值"""
189     # 检查必要列是否存在
190     required_columns = ['孕妇代码', '检测日期', '年龄']
191     for col in required_columns:
192         if col not in df.columns:
193             raise ValueError(f"数据框中缺少必要的列: {col}")
194
195     df_fixed = df.copy()
196     df_fixed['检测日期'] = pd.to_datetime(df_fixed['检测日期'])
197     original_ages = df_fixed['年龄'].copy()
198
199     # 按孕妇代码分组, 并在组内按检测日期排序
200     df_fixed = df_fixed.sort_values(by=['孕妇代码', '检测日期'])
201
202     # 存储修改报告
203     report = {
204         'total_changed': 0,
205         'changed_mothers': 0,
206         'changes_by_mother': {},

```

```

207     'changed_records': []
208 }
209
210 # 定义处理函数
211 def fix_group(group):
212     group = group.sort_values('检测日期')
213     ages = group['年龄'].values
214     dates = group['检测日期'].values
215     mother_id = group['孕妇代码'].iloc[0]
216
217     changes_in_group = []
218
219     for i in range(1, len(ages)):
220         if ages[i] < ages[i - 1]:
221             original_age = ages[i]
222             ages[i] = ages[i - 1]
223
224             # 记录修改
225             changes_in_group.append({
226                 'index': group.index[i],
227                 'date': dates[i],
228                 'original_age': original_age,
229                 'new_age': ages[i - 1]
230             })
231
232     group['年龄'] = ages
233
234     # 如果有修改, 添加到报告
235     if changes_in_group:
236         report['changes_by_mother'][mother_id] = changes_in_group
237         report['changed_records'].extend(changes_in_group)
238
239     return group
240
241 # 应用修复函数
242 df_fixed = df_fixed.groupby('孕妇代码').apply(fix_group)
243 df_fixed = df_fixed.reset_index(drop=True)
244
245 # 统计修改信息
246 report['total_changed'] = len(report['changed_records'])
247 report['changed_mothers'] = len(report['changes_by_mother'])
248
249 print(f"年龄序列修改报告:")
250 print(f"- 总共修改了 {report['total_changed']} 条记录")
251 print(f"- 涉及 {report['changed_mothers']} 位孕妇")
252
253 return df_fixed, report
254
255
256 if __name__ == "__main__":
257     # 读取数据
258     file_path = r"D:\pythonProject\数模准备\模型代码\datas\附件.xlsx"
259     df = pd.read_excel(file_path)
260
261     df_processed = preprocess_nipt_data(df)
262
263     df_fixed_production, production_changed_count = fix_production_count_with_stats(df_processed)
264     df_final, age_report = fix_age_sequence_detailed(df_fixed_production)
265
266     output_path = r"D:\pythonProject\数模准备\模型代码\datas\df_fixed.xlsx"

```



```

267     df_final.to_excel(output_path, index=False)
268
269     print(f"处理后的数据已保存至: {output_path}")
270     print(f"最终数据形状: {df_final.shape}")
271

```

附录 3

Q1 分组 try.py

对 BMI 和孕周各自分组后拟合趋势--相关性，后检验变量显著性

```

001 '''
002 Q1 最终解法: 对 BMI 和孕周各自分组后拟合趋势--相关性
003 给出等距分组, 等百分比分组, 临床经验分组, 效应变化分组和 Kmeans
004 分组这几种方法
005 通过加权 R^2 指标选择最佳方法, 分组后检验变量显著性
006 '''
007 import pandas as pd
008 import numpy as np
009 import statsmodels.api as sm
010 from statsmodels.formula.api import ols
011 import matplotlib.pyplot as plt
012 import seaborn as sns
013 from scipy import stats
014 from scipy.stats import fisher_exact
015 from sklearn.cluster import KMeans
016 import warnings
017
018 warnings.filterwarnings('ignore')
019
020 # 设置中文字体支持和忽略警告
021 plt.rcParams['font.sans-serif'].insert(0, 'SimHei')
022 plt.rcParams['axes.unicode_minus'] = False
023 import matplotlib
024
025 matplotlib.use('TkAgg')
026
027
028 def get_all_grouping_methods(df, variable, target='Y 染色
029 体浓度'):
030     """
031     尝试所有分组方法并返回最佳方法
032     """
033     methods = {}
034
035     # 等距分组
036     try:
037         methods['等距分组'] = pd.cut(df[variable],
038 bins=5, duplicates='drop')
039     except:
040         pass
041
042     # 等百分位数分组
043     try:
044         methods['等百分位数分组'] =

```

```

045 pd.qcut(df[variable], q=5, duplicates='drop')
046     except:
047         pass
048
049     # 临床标准分组
050     if variable == '孕妇 BMI':
051         bins = [0, 20, 28, 32, 36, 40, 100]#根据题
052 目信息
053         labels = ['<20', '20-28', '28-32', '32-36',
054 '36-40', '>40']
055         methods['临床标准分组'] =
056 pd.cut(df[variable], bins=bins, labels=labels)
057     elif variable == '孕周':
058         bins = [0, 13, 27, 45]#根据题目信息
059         labels = ['(<13) 早孕期', '[13,28) 中孕期',
060 '>27 晚孕期']
061         methods['临床标准分组'] =
062 pd.cut(df[variable], bins=bins, labels=labels)
063
064     # 效应变化趋势分组 - 基于目标变量的分位数
065     try:
066         methods['效应变化趋势分组'] =
067 pd.qcut(df[target], q=5, duplicates='drop')
068     except:
069         pass
070
071     # K-means 聚类分组
072     try:
073         kmeans = KMeans(n_clusters=5,
074 random_state=42, n_init=10)
075         kmeans.fit(df[[variable]])
076         methods['K-means 聚类分组'] =
077 pd.Series(kmeans.labels_, index=df.index).astype(str)
078     except:
079         pass
080
081     return methods
082
083
084 def evaluate_grouping_methods(df, grouping_variable,
085 target='V 染色体浓度', control_variable=None):
086     """
087     评估所有分组方法并选择最优
088     """
089     methods = get_all_grouping_methods(df,
090 grouping_variable, target)
091     results = {}
092
093     for method_name, groups in methods.items():
094         try:
095             # 确保分组有效
096             if groups.isna().all():
097                 continue
098
099             # 对每个分组进行回归分析
100             r_squared_values = []
101             sample_sizes = []
102
103             for group_name in groups.unique():
104                 group_data = df[groups ==

```

```

105 group_name]
106         if len(group_data) < 10: #
107             确保有足够的样本
108             continue
109
110         # 构建回归公式
111         if control_variable:
112             formula = f"{target} ~
113 {grouping_variable} + {control_variable}"
114         else:
115             formula = f"{target} ~
116 {grouping_variable}"
117
118         try:
119             model = ols(formula,
120 data=group_data).fit()#拟合回归
121
122 r_squared_values.append(model.rsquared)
123
124 sample_sizes.append(len(group_data))
125         except:
126             continue
127
128         if not r_squared_values:
129             continue
130
131         # 计算加权平均 R²
132         total_samples = sum(sample_sizes)
133         weighted_r2 = sum([r2 * n for r2, n
134 in zip(r_squared_values, sample_sizes)]) / total_samples
135         avg_r2 = np.mean(r_squared_values)
136
137         results[method_name] = {
138             'weighted_r2': weighted_r2,
139             'avg_r2': avg_r2,
140             'num_groups':
141 len(groups.unique()),
142             'total_samples':
143 total_samples
144         }
145     except Exception as e:
146         print(f"方法 {method_name} 评估失败:
147 {e}")
148         continue
149
150     # 选择 R²最高的方法
151     if results:
152         best_method = max(results.items(),
153 key=lambda x: x[1]['weighted_r2'])
154         return best_method[0], results, methods
155     else:
156         return None, {}, {}
157
158 # 可视化分组过程和结果
159 def visualize_grouping_process(df, variable, groups,
160 target='Y 染色体浓度'):
161
162     fig, axes = plt.subplots(2, 2, figsize=(15, 12))
163
164     # 绘制原始数据散点图

```

```

165     axes[0, 0].scatter(df[variable], df[target],
166 alpha=0.5)
167     axes[0, 0].set_xlabel(variable)
168     axes[0, 0].set_ylabel(target)
169     axes[0, 0].set_title(f'{variable}与{target}的散点
170 图')
171
172     # 绘制分组箱线图
173     group_data = []
174     group_names = []
175     for group_name in sorted(groups.unique()):
176         group_values = df[groups ==
177 group_name][target]
178         if len(group_values) > 0:
179             group_data.append(group_values)
180
181     group_names.append(str(group_name))
182
183     axes[0, 1].boxplot(group_data, labels=group_names)
184     axes[0, 1].set_xlabel('分组')
185     axes[0, 1].set_ylabel(target)
186     axes[0, 1].set_title(f'按{variable}分组后的
187 {target}分布')
188     plt.setp(axes[0, 1].xaxis.get_majorticklabels(),
189 rotation=45)
190
191     # 计算并显示组内相关系数
192     try:
193         from scipy.stats import f_oneway
194         f_stat, p_value = f_oneway(*group_data)
195         axes[1, 0].text(0.1, 0.8, f'ANOVA F 值:
196 {f_stat:.3f}\nP 值: {p_value:.3e}',
197
198 transform=axes[1, 0].transAxes, fontsize=12,
199
200 bbox=dict(boxstyle="round,pad=0.3",
201 facecolor="lightgray"))
202     except:
203         pass
204
205     axes[1, 0].axis('off')
206     axes[1, 0].set_title('组间差异检验')
207
208     # 显示分组统计信息
209     stats_text = "分组统计:\n"
210     for i, group_name in enumerate(group_names):
211         group_values = group_data[i]
212         stats_text += f"组 {group_name}:
213 N={len(group_values)}, 均值={group_values.mean():.2f}, 标
214 准差={group_values.std():.2f}\n"
215
216     axes[1, 1].text(0.1, 0.1, stats_text,
217 transform=axes[1, 1].transAxes, fontsize=10,
218
219 verticalalignment='bottom',
220 bbox=dict(boxstyle="round,pad=0.3",
221 facecolor="lightblue"))
222     axes[1, 1].axis('off')
223     axes[1, 1].set_title('分组统计信息')
224

```

```

225     plt.tight_layout()
226     plt.show()
227
228
229 # 绘制所有分组的散点图和回归线，放在同一个布局中
230 def plot_all_scatter_with_regression(df, x_var, y_var,
231 groups, title_prefix=""):
232     unique_groups = groups.unique()
233     n_groups = len(unique_groups)
234
235     # 计算子图布局
236     n_cols = min(3, n_groups)
237     n_rows = (n_groups + n_cols - 1) // n_cols
238
239     # 创建子图
240     fig, axes = plt.subplots(n_rows, n_cols, figsize=(5
241 * n_cols, 4 * n_rows))
242     if n_rows == 1 and n_cols == 1:
243         axes = np.array([axes])
244     axes = axes.flatten()
245
246     # 为每个分组绘制散点图和回归线
247     for i, group_name in enumerate(unique_groups):
248         if i >= len(axes):
249             break
250
251         group_data = df[groups == group_name]
252         if len(group_data) < 5:
253             axes[i].text(0.5, 0.5, f"分组
254 {group_name}\n 样本量不足",
255
256 ha='center', va='center', transform=axes[i].transAxes)
257             axes[i].set_title(f'分组:
258 {group_name}')
259             continue
260
261         axes[i].scatter(group_data[x_var],
262 group_data[y_var], alpha=0.5)
263
264         # 计算回归线
265         try:
266             z = np.polyfit(group_data[x_var],
267 group_data[y_var], 1)
268             p = np.poly1d(z)
269             axes[i].plot(group_data[x_var],
270 p(group_data[x_var]), "r--", alpha=0.8)
271
272             # 计算 R²
273             y_pred = p(group_data[x_var])
274             y_true = group_data[y_var]
275             r2 = 1 - ((y_true - y_pred) ** 2).sum()
276 / ((y_true - y_true.mean()) ** 2).sum()
277
278             # 添加方程和 R² 文本
279             equation = f'y = {z[0]:.3f}x +
280 {z[1]:.3f}'
281             axes[i].text(0.05, 0.95,
282 f'{equation}\n$R^2$ = {r2:.3f}',
283
284 transform=axes[i].transAxes, fontsize=10,

```

```

285
286 bbox=dict(boxstyle="round", facecolor="wheat",
287 alpha=0.8),
288
289 verticalalignment='top')
290         except Exception as e:
291             axes[i].text(0.5, 0.5, f"回归失败:
292 {str(e)}",
293
294 ha='center', va='center', transform=axes[i].transAxes)
295
296         axes[i].set_xlabel(x_var)
297         axes[i].set_ylabel(y_var)
298         axes[i].set_title(f'分组: {group_name}')
299
300     for i in range(n_groups, len(axes)):
301         axes[i].set_visible(False)
302
303     plt.suptitle(f'{{title_prefix}}{x_var} 与 {y_var} 的
304 关系', fontsize=16)
305     plt.tight_layout(rect=[0, 0, 1, 0.96])
306     plt.show()
307
308 # 对分组分析结果进行显著性检验
309 def significance_test(df, grouping_variable, groups,
310 target='Y 染色体浓度', control_variable=None):
311     significant_effects = []
312     p_values = []
313     group_names = []
314
315     # 先进行整体回归分析
316     try:
317         if control_variable:
318             formula = f"{{target}} ~
319 {{grouping_variable}} + {{control_variable}}"
320         else:
321             formula = f"{{target}} ~
322 {{grouping_variable}}"
323
324         overall_model = ols(formula, data=df).fit()
325         overall_p =
326 overall_model.pvalues[grouping_variable]
327         overall_r2 = overall_model.rsquared
328     except Exception as e:
329         print(f"整体回归分析失败: {e}")
330         overall_p = np.nan
331         overall_r2 = np.nan
332
333     # 进行分组回归分析
334     for group_name in groups.unique():
335         group_data = df[groups == group_name]
336         if len(group_data) < 10:
337             continue
338
339         # 构建回归公式
340         if control_variable:
341             formula = f"{{target}} ~
342 {{grouping_variable}} + {{control_variable}}"
343         else:
344             formula = f"{{target}} ~

```

```

345 {grouping_variable}"
346
347         try:
348             model = ols(formula,
349 data=group_data).fit()
350             p_value =
351 model.pvalues[grouping_variable]
352             p_values.append(p_value)
353             significant_effects.append(p_value
354 < 0.05)
355
356 group_names.append(str(group_name))
357         except:
358             continue
359
360     if not p_values:
361         print("没有足够的数据进行显著性检验")
362         return None, None, None, None
363
364     # 进行 Fisher 组合检验
365     try:
366         chi2_stat, combined_p =
stats.combine_pvalues(p_values, method='fisher')
367     except:
368         combined_p = np.nan
369
370     for i, group_name in enumerate(group_names):
371         print(f"分组 {group_name}: p 值 =
{p_values[i]:.4e} {'(显著)' if significant_effects[i] else
'(不显著)'}")
372
373     return significant_effects, p_values, overall_p,
combined_p
374
375 def main():
376     data = pd.read_excel(r"D:\pythonProject\数模准备\
模型代码\datas\processed_data1(2).xlsx")
377     df = pd.DataFrame(data, columns=['孕妇代码', '孕周
', '孕妇 BMI', 'Y 染色体浓度'])
378
379     best_method_bmi, results_bmi, methods_bmi =
evaluate_grouping_methods(
380         df, '孕妇 BMI', target='Y 染色体浓度',
381         control_variable='孕周'
382     )
383
384     if best_method_bmi:
385         print(f"最优 BMI 分组方法:
{best_method_bmi}")
386         bmi_groups = methods_bmi[best_method_bmi]
387
388         # 可视化分组过程
389         visualize_grouping_process(df, '孕妇 BMI',
bmi_groups)
390
391         # 绘制所有分组的散点图和回归线
392         plot_all_scatter_with_regression(

```

```

        df, '孕周', 'Y 染色体浓度',
bmi_groups,
        title_prefix=f"BMI 分组
({best_method_bmi}): "
    )

    # 进行显著性检验
    sig_effects_bmi, p_values_bmi,
overall_p_bmi, combined_p_bmi = significance_test(
        df, '孕周', bmi_groups,
control_variable='孕妇 BMI'
    )

    else:
        print("没有找到有效的 BMI 分组方法")

    # 按孕周分组分析 BMI 效应
    best_method_ga, results_ga, methods_ga =
evaluate_grouping_methods(
        df, '孕周', target='Y 染色体浓度',
control_variable='孕妇 BMI'
    )

    if best_method_ga:
        print(f"最优孕周分组方法: {best_method_ga}")
        ga_groups = methods_ga[best_method_ga]

        # 可视化分组过程
        visualize_grouping_process(df, '孕周',
ga_groups)

        # 绘制所有分组的散点图和回归线
        plot_all_scatter_with_regression(
            df, '孕妇 BMI', 'Y 染色体浓度',
ga_groups,
            title_prefix=f"孕周分组
({best_method_ga}): "
        )

        # 进行显著性检验
        sig_effects_ga, p_values_ga, overall_p_ga,
combined_p_ga = significance_test(
            df, '孕妇 BMI', ga_groups,
control_variable='孕周'
        )

    else:
        print("没有找到有效的孕周分组方法")

if __name__ == "__main__":
    main()

```

附录 4

Q2 求解函数.py

使用半参数 cox 模型，求解 BMI 在 Y 染色体浓度达到 4%时的 t_ideal


```

001 '''
002 求解 cox 模型, 拟合 bmi-t_ideal 关系
003 灵敏性分析
004 '''
005 import os
006
007 import pandas as pd
008 import numpy as np
009 from lifelines import CoxPHFitter, WeibullAFTFitter
010 from lifelines.utils import concordance_index
011 from sklearn.preprocessing import PolynomialFeatures
012 import matplotlib.pyplot as plt
013 from scipy import stats
014 plt.rcParams['font.sans-serif'].insert(0, 'SimHei')
015 plt.rcParams['axes.unicode_minus'] = False
016 import matplotlib
017 matplotlib.use('TkAgg')
018
019 processed_df=pd.read_excel(r"D:\xwechat_files\wxid_hrp73h5vqmjx22_24f8\msg\file\2025-09\processed_data1(2).xlsx")
020
021
022 def prepare_enhanced_cox_data(df):
023     """
024     准备增强的 Cox 模型数据, 包含更多特征
025     """
026     cox_data = df.copy()
027
028     # 创建事件变量 (Y 浓度达标 4%)
029     cox_data['event'] = (cox_data['Y 染色体浓度'] >= 0.0395).astype(float)
030
031     result_data = []
032
033     for pregnant_id in cox_data['孕妇代码'].unique():
034         woman_data = cox_data[cox_data['孕妇代码'] == pregnant_id].sort_values('孕周')
035
036         # 计算初始 Y 染色体浓度 (第一次检测)
037         initial_concentration = woman_data['Y 染色体浓度'].iloc[0]
038         initial_week = woman_data['孕周'].iloc[0]
039
040         # 找到首次达到 4% 的记录
041         reached_4_percent = woman_data[woman_data['Y 染色体浓度'] >= 0.0395]
042
043         if len(reached_4_percent) > 0:
044             # 事件发生
045             first_reach = reached_4_percent.iloc[0]
046             result_data.append({
047                 '孕妇代码': pregnant_id,
048                 'duration': first_reach['孕周'], # 事件时间
049                 'event': 1, # 事件发生
050                 'BMI': first_reach['孕妇 BMI'],
051                 '年龄': first_reach['年龄'],
052                 '初始浓度': initial_concentration,
053                 '初始孕周': initial_week,
054                 'IVF': 1 if first_reach['IVF 妊娠'] == 'IVF' else 0,
055                 '怀孕次数': first_reach['怀孕次数'],
056                 '生产次数': first_reach['生产次数']
057             })
058         else:
059             # 事件未发生 (删失)
060             last_record = woman_data.iloc[-1]

```

```

061         result_data.append({
062             '孕妇代码': pregnant_id,
063             'duration': last_record['孕周'], # 删失时间
064             'event': 0, # 事件未发生
065             'BMI': last_record['孕妇 BMI'],
066             '年龄': last_record['年龄'],
067             '初始浓度': initial_concentration,
068             '初始孕周': initial_week,
069             'IVF': 1 if last_record['IVF 妊娠'] == 'IVF' else 0,
070             '怀孕次数': last_record['怀孕次数'],
071             '生产次数': last_record['生产次数']
072         })
073
074     return pd.DataFrame(result_data)
075
076
077 cox_df = prepare_enhanced_cox_data(processed_df)
078
079 typical_initial_week = cox_df['初始孕周'].median()
080 typical_initial_concentration = cox_df['初始浓度'].median()
081
082 def fit_cox_model(cox_df):
083
084
085     cox_df['BMI_centered'] = cox_df['BMI'] - cox_df['BMI'].mean()
086     cox_df['BMI_centered_squared'] = cox_df['BMI_centered'] ** 2
087     cox_df['BMI_centered_cubed'] = cox_df['BMI_centered'] ** 3
088
089     cox_df['log_初始浓度'] = np.log(cox_df['初始浓度'] + 0.001)
090
091     # 尝试不同的模型配置
092     formulas = [
093         "BMI_centered_cubed + 初始孕周"
094         "BMI_centered + BMI_centered_squared",
095         "BMI_centered**2 + BMI",
096         "BMI_centered + BMI_centered_squared+ log_初始浓度+ 初始孕周",
097         "BMI_centered_cubed + 初始孕周"
098     ]
099
100     best_cph = None
101     best_c_index = 0
102     best_formula = ""
103
104     for formula in formulas:
105         try:
106             cph = CoxPHFitter()
107             cph.fit(cox_df, duration_col='duration', event_col='event', formula=formula)
108
109             if cph.concordance_index_ > best_c_index:
110                 best_c_index = cph.concordance_index_
111                 best_cph = cph
112                 best_formula = formula
113
114             print(f"Formula: {formula}")
115             print(f"C-index: {cph.concordance_index_:.3f}")
116             print("-" * 50)
117         except:
118             continue
119
120     print(best_cph.summary)

```

```

121
122     return best_cph, cox_df
123
124
125 fit_cox_model(cox_df)
126
127 best_cph, cox_df_featured = fit_cox_model(cox_df)
128
129 #-----可视化生存函数-----
130
131 def display_full_model_function(cph_model):
132     # 构建公式字符串
133     formula_str = "exp("
134     for name, param_val in cph_model.params_.items():
135         formula_str += f"({param_val:.4f} * {name}) + "
136     formula_str = formula_str[:-3] + ")" # 去掉最后的 " + "
137     print("具体表达式为:", formula_str)
138
139     # 可视化基准生存函数
140     plt.figure(figsize=(8, 5))
141     cph_model.baseline_survival_.plot(title="基准生存函数  $S_0(t)$ ")
142     plt.ylabel("基准生存概率 (事件未发生)")
143     plt.xlabel("孕周 (t)")
144     plt.grid(True, linestyle='--', alpha=0.6)
145     plt.show()
146
147
148 display_full_model_function(best_cph)
149
150 #-----封装求解模型-----
151
152 def solve_for_ideal_time(bmi_value, cph_model, df_featured, typical_week, typical_conc):
153     """
154     为给定的 BMI 值, 求解 95%事件达标的时点  $T_{ideal}$ .
155     """
156     bmi_mean = df_featured['BMI'].mean()
157     X_pred = pd.DataFrame({
158         'BMI_centered': [bmi_value - bmi_mean],
159         'BMI_centered_squared': [(bmi_value - bmi_mean)**2],
160         'BMI_centered_cubed': [(bmi_value - bmi_mean) **3 ],
161         'log_初始浓度': [np.log(typical_conc + 1e-6)],
162         '初始孕周': [typical_week]
163     })
164     # p=0.05 对应 95% 事件发生率
165     time_95 = cph_model.predict_percentile(X_pred, p=0.05)
166     return time_95
167
168 bmi_range = np.arange(cox_df['BMI'].min(), cox_df['BMI'].max(), 0.1)
169 ideal_times = [solve_for_ideal_time(b, best_cph, cox_df, typical_initial_week, typical_initial_concentration) for b
170 in bmi_range]
171
172 # 创建新的 (BMI,  $T_{ideal}$ ) 数据集
173 bmi_time_df = pd.DataFrame({
174     'BMI': bmi_range,
175     'T_ideal': ideal_times
176 })
177
178
179
180 save_dir = r"/result"

```

```

181 os.makedirs(save_dir, exist_ok=True) # 自动创建目录（如果不存在）
182
183 # 2. 保存数据集到 Excel
184 file_path = os.path.join(save_dir, "bmi_time 低.xlsx")
185 bmi_time_df.to_excel(file_path, index=False) # index=False 避免保存行索引
186 print(f"数据集已成功保存至: {file_path}")
187
188
189 #-----//////灵敏度分析
190
191 def sensitivity_analysis(cox_df, test_size=0.2, random_state=42):
192     from sklearn.model_selection import train_test_split
193
194     train_df, test_df = train_test_split(
195         cox_df,
196         test_size=test_size,
197         random_state=random_state,
198         stratify=cox_df['event'] # 保持事件比例一致
199     )
200
201     # 对训练集进行特征工程
202     train_df = train_df.copy()
203     train_df['BMI_centered'] = train_df['BMI'] - train_df['BMI'].mean()
204     train_df['BMI_centered_squared'] = train_df['BMI_centered'] ** 2
205     train_df['BMI_centered_cubed'] = train_df['BMI_centered'] ** 3
206     train_df['log_初始浓度'] = np.log(train_df['初始浓度'] + 0.001)
207     train_df['年龄_centered'] = train_df['年龄'] - train_df['年龄'].mean()
208
209     # 使用与之前相同的公式训练模型
210     formula = "BMI_centered_cubed + 初始孕周"
211
212     cph_train = CoxPHFitter()
213     cph_train.fit(train_df, duration_col='duration', event_col='event', formula=formula)
214
215     print(cph_train.summary)#训练集模型性能
216
217     test_df = test_df.copy()
218     test_df['BMI_centered'] = test_df['BMI'] - train_df['BMI'].mean()
219     test_df['BMI_centered_squared'] = test_df['BMI_centered'] ** 2
220     test_df['BMI_centered_cubed'] = test_df['BMI_centered'] ** 3
221     test_df['log_初始浓度'] = np.log(test_df['初始浓度'] + 0.001)
222     test_df['年龄_centered'] = test_df['年龄'] - train_df['年龄'].mean()
223
224     try:
225         test_c_index = cph_train.score(test_df, scoring="concordance_index")
226         print(f"\n 测试集 C-index: {test_c_index:.3f}")
227         c_index_diff = abs(cph_train.concordance_index_ - test_c_index)
228         print(f"C-index 差异: {c_index_diff:.3f}")
229
230         if c_index_diff < 0.05:
231             print("模型表现稳定, 过拟合风险较低")
232         else:
233             print("模型可能存在过拟合风险")
234
235     except Exception as e:
236         print(f"测试集评估出错: {e}")
237
238     return cph_train, train_df, test_df
239
240

```

```

241 # 执行敏感性分析
242 cph_trained, train_data, test_data = sensitivity_analysis(cox_df)
243
244
245 # 交叉验证评估模型稳定性
246 def cross_validate_cox(cox_df, n_splits=5, random_state=42):
247
248     from sklearn.model_selection import KFold
249     from lifelines.utils import concordance_index
250
251     cox_df_processed = cox_df.copy()
252     cox_df_processed['BMI_centered'] = cox_df_processed['BMI'] - cox_df_processed['BMI'].mean()
253     cox_df_processed['BMI_centered_squared'] = cox_df_processed['BMI_centered'] ** 2
254     cox_df_processed['BMI_centered_cubed'] = cox_df_processed['BMI_centered'] ** 3
255     cox_df_processed['log_初始浓度'] = np.log(cox_df_processed['初始浓度'] + 0.001)
256     cox_df_processed['年龄_centered'] = cox_df_processed['年龄'] - cox_df_processed['年龄'].mean()
257
258     # 设置 KFold 交叉验证
259     kf = KFold(n_splits=n_splits, shuffle=True, random_state=random_state)
260     c_indices = []
261
262     for train_index, test_index in kf.split(cox_df_processed):
263         train_fold = cox_df_processed.iloc[train_index]
264         test_fold = cox_df_processed.iloc[test_index]
265
266         # 训练模型
267         cph_fold = CoxPHFitter()
268         cph_fold.fit(train_fold, duration_col='duration', event_col='event',
269                     formula="BMI_centered_cubed + 初始孕周")
270
271         # 在测试集上预测风险分数
272         test_fold['predicted_risk'] = cph_fold.predict_partial_hazard(test_fold)
273
274         c_index = concordance_index(
275             test_fold['duration'],
276             -test_fold['predicted_risk'],
277             test_fold['event']
278         )
279         c_indices.append(c_index)
280
281     # 输出交叉验证结果
282     print(f"\n{'-' * 50}")
283     print(f"{n_splits}折交叉验证结果:")
284     print(f"C-index 平均值: {np.mean(c_indices):.3f}")
285     print(f"C-index 标准差: {np.std(c_indices):.3f}")
286     print(f"C-index 范围: [{min(c_indices):.3f}, {max(c_indices):.3f}]")
287
288     return c_indices
289
290
291 # 执行交叉验证
292 cv_results = cross_validate_cox(cox_df)
293
294 # 绘制 C-index 分布
295 plt.figure(figsize=(8, 5))
296 plt.boxplot(cv_results)
297 plt.title('交叉验证 C-index 分布')
298 plt.ylabel('C-index')
299 plt.grid(True, alpha=0.3)
300 plt.show()

```

```

301
302
303 # 评估模型在不同 BMI 分组上的表现
304 def evaluate_by_bmi_groups(model, test_df, n_groups=3):
305
306     test_df = test_df.copy()
307     test_df['BMI_group'] = pd.qcut(test_df['BMI'], n_groups, labels=[f'Group {i + 1}' for i in range(n_groups)])
308
309     group_c_indices = {}
310
311     for group in test_df['BMI_group'].unique():
312         group_data = test_df[test_df['BMI_group'] == group]
313
314         # 预测风险分数
315         group_data['predicted_risk'] = model.predict_partial_hazard(group_data)
316
317         # 计算该组的 C-index
318         c_index = concordance_index(
319             group_data['duration'],
320             -group_data['predicted_risk'],
321             group_data['event']
322         )
323         group_c_indices[group] = c_index
324
325     # 输出结果
326     print(f"\n 按 BMI 分组评估结果 ({n_groups}组):")
327     for group, c_index in group_c_indices.items():
328         print(f"{group}: C-index = {c_index:.3f}")
329
330     return group_c_indices
331
332
333 # 执行分组评估
334 bmi_group_results = evaluate_by_bmi_groups(cph_trained, test_data)
335
336
337 #print("\n 灵敏性分析完成!")

```

附录 5

Q2 动态规划.py

定义风险函数，以及总成本目标 运用动态规划思想同时优化分组和最佳时点

```

001 '''
002 定义风险函数，以及总成本目标
003 运用动态规划思想同时优化分组和最佳时点
004 通过网格搜索λ，以 Pareto 前沿调参
005 '''
006
007 import numpy as np
008 import pandas as pd
009 import matplotlib.pyplot as plt
010 import warnings
011
012 warnings.filterwarnings('ignore')
013 import matplotlib
014
015 matplotlib.use('TkAgg')
016 plt.rcParams['font.sans-serif'].insert(0, 'SimHei')
017 plt.rcParams['axes.unicode_minus'] = False

```

```

018
019 # 读取聚类结果数据
020 #df1 = pd.read_excel(r"D:\pythonProject\数模准备\模型代码
021 \result\bmi_time1.xlsx")#全集
022 df=pd.read_excel(r"D:\pythonProject\数模准备\模型代码\result\bmi_time
023 低.xlsx")#检测误差稳定数据集
024
025 # 定义风险函数
026 def risk_function(t, t_ideal):
027     if t > t_ideal:
028         A = 0.83
029     elif t < t_ideal:
030         A = 0.27
031     else: # t == t_ideal
032         A = 0 # 当t等于t_ideal时, 风险为0
033     return (t - t_ideal) ** 2 * A
034
035
036 # 计算一组数据的最佳T和最小风险和
037 def compute_group_cost(t_ideals, lambda_val):
038     if len(t_ideals) == 0:
039         return 0, 0, 0
040     # 排序以便计算
041     sorted_t = sorted(t_ideals)
042     n = len(sorted_t)
043     prefix_sum = np.cumsum(sorted_t)
044     prefix_sum_sq = np.cumsum(np.array(sorted_t) ** 2)
045     total_sum = prefix_sum[-1]
046     total_sum_sq = prefix_sum_sq[-1]
047     # 初始化最小风险和最佳T
048     min_risk = float('inf')
049     best_T = None
050     # 检查所有临界点 (每个t_ideal)
051     for i, T_candidate in enumerate(sorted_t):
052         # 计算分割点 (小于等于T_candidate的样本数)
053         n1 = i + 1
054         n2 = n - n1
055         S1 = prefix_sum[i] if i >= 0 else 0
056         S2 = total_sum - S1
057         sum_sq1 = prefix_sum_sq[i] if i >= 0 else 0
058         sum_sq2 = total_sum_sq - sum_sq1
059         # 计算风险和
060         risk = 0.27 * (n1 * T_candidate ** 2 - 2 * T_candidate
061 * S1 + sum_sq1) + \
062             0.83 * (n2 * T_candidate ** 2 - 2 *
063 T_candidate * S2 + sum_sq2)
064         if risk < min_risk:
065             min_risk = risk
066             best_T = T_candidate
067     # 检查所有区间 (相邻t_ideal之间)
068     for i in range(n - 1):
069         # 计算区间内的极值点
070         n1 = i + 1
071         n2 = n - n1
072         S1 = prefix_sum[i]
073         S2 = total_sum - S1
074         denominator = 0.27 * n1 + 0.83 * n2
075         if denominator == 0:
076             continue
077         T0 = (0.27 * S1 + 0.83 * S2) / denominator

```

```

078         # 检查 T0 是否在当前区间内
079         if sorted_t[i] <= T0 < sorted_t[i + 1]:
080             # 计算风险和
081             risk = 0.27 * (n1 * T0 ** 2 - 2 * T0 * S1 +
082 prefix_sum_sq[i]) + \
083                 0.83 * (n2 * T0 ** 2 - 2 * T0 * S2
084 + (total_sum_sq - prefix_sum_sq[i]))
085             if risk < min_risk:
086                 min_risk = risk
087                 best_T = T0
088             # 检查最后一个区间 (大于所有 t_ideal)
089             # 所有样本都小于 T, 风险函数为 0.27*(T - t_ideal)^2
090             # 最小值在 T = 所有 t_ideal 的均值
091             T_mean = total_sum / n
092             if T_mean >= sorted_t[-1]:
093                 risk = 0.27 * (n * T_mean ** 2 - 2 * T_mean * total_sum
094 + total_sum_sq)
095                 if risk < min_risk:
096                     min_risk = risk
097                     best_T = T_mean
098             # 计算总成本 (风险和 + λ*T)
099             cost = min_risk + lambda_val * best_T
100             return min_risk, best_T, cost
101
102
103 # 动态规划主函数
104 def optimal_partitioning(BMI, T_ideal, lambda_val):
105     N = len(BMI)
106     if N == 0:
107         return [], [], 0, 0, 0
108     # 初始化 DP 数组
109     dp = np.full(N + 1, np.inf) # dp[i] = 前 i 个点的最小成本
110     dp[0] = 0 # 0 个点成本为 0
111     split_points = [0] * (N + 1) # 记录分割点
112     # 动态规划主循环
113     for i in range(1, N + 1): # 考虑前 i 个点
114         for j in range(0, i): # 尝试所有分割点 j
115             # 计算组[j, i-1]的成本
116             group_t_ideals = T_ideal[j:i]
117             _, _, cost_val =
118 compute_group_cost(group_t_ideals, lambda_val)
119             if dp[j] + cost_val < dp[i]:
120                 dp[i] = dp[j] + cost_val
121                 split_points[i] = j
122     # 回溯分组结果
123     groups = []
124     current = N
125     while current > 0:
126         prev = split_points[current]
127         groups.append((prev, current - 1)) # 组[prev,
128 current-1]
129         current = prev
130     groups.reverse()
131     # 计算每个组的最佳 T 和总风险、总 T
132     group_Ts = []
133     total_risk = 0
134     total_T = 0
135     for (start, end) in groups:
136         group_t_ideals = T_ideal[start:end + 1]
137         risk, T_opt, _ = compute_group_cost(group_t_ideals, 0)

```



```

138         group_Ts.append(T_opt)
139         total_risk += risk
140         total_T += T_opt
141     total_cost = dp[N]
142     return groups, group_Ts, total_risk, total_T, total_cost
143
144
145 # 可视化分组结果
146 def plot_partitioning(BMI, T_ideal, groups, group_Ts, title="BMI 分组
147 结果"):
148     plt.figure(figsize=(12, 8))
149     colors = plt.cm.tab10(np.linspace(0, 1, len(groups)))
150     for i, (start, end) in enumerate(groups):
151         # 绘制组内点
152         plt.scatter(BMI[start:end + 1], T_ideal[start:end + 1],
153                     color=colors[i], label=f'组 {i +
154 1}', s=50, alpha=0.7)
155         # 绘制组内最佳 T 线
156         plt.axhline(y=group_Ts[i], color=colors[i],
157                     linestyle='--', alpha=0.7)
158         # 标记最佳 T 值
159         mid_idx = (start + end) // 2
160         plt.text(BMI[mid_idx], group_Ts[i],
161                  f'T={group_Ts[i]:.2f}',
162                  color=colors[i], fontsize=10,
163                  ha='center')
164         # 添加分组边界线
165         for i in range(1, len(groups)):
166             boundary_bmi = (BMI[groups[i][0] - 1] +
167                             BMI[groups[i][0]]) / 2
168             plt.axvline(x=boundary_bmi, color='gray',
169                         linestyle=':', alpha=0.5)
170         plt.xlabel('BMI')
171         plt.ylabel('T_ideal')
172         plt.title(title)
173         plt.legend()
174         plt.grid(True, alpha=0.3)
175         plt.tight_layout()
176         plt.show()
177
178
179 # 参数调优函数 (Pareto 前沿分析)
180 def tune_lambda(BMI, T_ideal, lambda_values=None):
181     if lambda_values is None:
182         # 自动生成λ值 (对数间隔)
183         lambda_values = np.logspace(-2, 2, 20) # 从 0.01 到 100
184     results = []
185     for lambda_val in lambda_values:
186         groups, group_Ts, total_risk, total_T, total_cost =
187         optimal_partitioning(
188             BMI, T_ideal, lambda_val)
189         num_groups = len(groups)
190         avg_T = total_T / num_groups if num_groups > 0 else 0
191         results.append({
192             'lambda': lambda_val,
193             'total_risk': total_risk,
194             'total_T': total_T,
195             'avg_T': avg_T,
196             'total_cost': total_cost,
197             'num_groups': num_groups

```

```

198         })
199         results_df = pd.DataFrame(results)
200
201         # 选择最佳λ
202         results_df['norm_risk'] = (results_df['total_risk'] -
203 results_df['total_risk'].min()) / \
204
205 (results_df['total_risk'].max() - results_df['total_risk'].min())
206         results_df['norm_avg_T'] = (results_df['avg_T'] -
207 results_df['avg_T'].min()) / \
208
209 (results_df['avg_T'].max() - results_df['avg_T'].min())
210
211         results_df['score'] = 0.7 * results_df['norm_risk'] + 0.3 *
212 results_df['norm_avg_T']
213         best_idx = results_df['score'].idxmin()
214         best_lambda = results_df.loc[best_idx, 'lambda']
215
216
217         return results_df, best_lambda
218
219
220 # 主程序执行部分
221 if __name__ == "__main__":
222     # 参数调优
223     results_df, best_lambda = tune_lambda(df['BMI'].values,
224 df['T_ideal'].values)
225
226     # 使用最佳λ进行分组
227     groups, group_Ts, total_risk, total_T, total_cost =
228 optimal_partitioning(
229         df['BMI'].values, df['T_ideal'].values, best_lambda)
230
231     # 计算平均 T
232     num_groups = len(groups)
233     avg_T = total_T / num_groups if num_groups > 0 else 0
234
235     #输出展示结果
236     print("\n 分组结果:")
237     for i, (start, end) in enumerate(groups):
238         print(f"组 {i + 1}: BMI 范围
239 [{df['BMI'].iloc[start]:.2f}, {df['BMI'].iloc[end]:.2f}], "
240             f"样本数 {end - start + 1}, 最佳
241 T={group_Ts[i]:.2f}")
242
243     print(f"\n 总风险和: {total_risk:.2f}")
244     print(f"总 T 和: {total_T:.2f}")
245     print(f"平均 T: {avg_T:.2f}")
246     print(f"组数: {num_groups}")
247     print(f"总成本: {total_cost:.2f}")
248
249     # 可视化分组结果
250     plot_partitioning(df['BMI'].values, df['T_ideal'].values,
251 groups, group_Ts,
252
253                     title=f"BMI 分组结果
254 (λ={best_lambda:.4f}, 平均 T={avg_T:.2f})"

```

附录 6

Q3 特征选择.py

通过模型内置参数来评估特征重要性，参考模型拟合度进行特征选择

```
001 import pandas as pd
002 import numpy as np
003 from lifelines import CoxPHFitter
004 import matplotlib.pyplot as plt
005 import os
006
007 plt.rcParams['font.sans-serif'].insert(0, 'SimHei')
008 plt.rcParams['axes.unicode_minus'] = False
009 import matplotlib
010
011 matplotlib.use('TkAgg')
012
013 processed_df = pd.read_excel(r"D:\pythonProject\数模准备\
014 模型代码\datas\processed_dataE(2).xlsx")
015
016
017 def prepare_enhanced_cox_data(df):
018     cox_data = df.copy()
019     # 创建事件变量
020     cox_data['event'] = (cox_data['Y 染色体浓度'] >=
021 0.0395).astype(float)
022     result_data = []
023
024     for pregnant_id in cox_data['孕妇代码'].unique():
025         woman_data = cox_data[cox_data['孕妇代码']
026 == pregnant_id].sort_values('孕周')
027         initial_concentration = woman_data['Y 染色体
028 浓度'].iloc[0]
029         initial_week = woman_data['孕周'].iloc[0]
030
031         # 找到首次达到 4%的记录
032         reached_4_percent =
033 woman_data[woman_data['Y 染色体浓度'] >= 0.0395]
034
035         if len(reached_4_percent) > 0:
036             # 事件发生
037             first_reach =
038 reached_4_percent.iloc[0]
039             record = {
040                 '孕妇代码': pregnant_id,
041                 'duration': first_reach['孕
042 周'],
043                 'event': 1,
044                 'BMI': first_reach['孕妇
045 BMI'],
046                 '年龄': first_reach['年龄'],
047                 '初始浓度':
048 initial_concentration,
049                 '初始孕周': initial_week,
050                 '生产次数': first_reach['生产
051 次数'],
052                 '体重': first_reach.get('体
053 重', np.nan),
054                 '身高': first_reach.get('身
055 高', np.nan),
056                 '误差 E': first_reach.get('误
057 差 E', np.nan)
058             }
```

```

059         else:
060             # 事件未发生 (删除)
061             last_record = woman_data.iloc[-1]
062             record = {
063                 '孕妇代码': pregnant_id,
064                 'duration': last_record['孕
065 周'],
066                 'event': 0,
067                 'BMI': last_record['孕妇
068 BMI'],
069                 '年龄': last_record['年龄'],
070                 '初始浓度':
071 initial_concentration,
072                 '初始孕周': initial_week,
073                 '生产次数': last_record['生产
074 次数'],
075                 '体重': last_record.get('体
076 重', np.nan),
077                 '身高': last_record.get('身
078 高', np.nan),
079                 '误差 E': last_record.get('误
080 差 E', np.nan)
081             }
082             result_data.append(record)
083
084         return pd.DataFrame(result_data)
085
086
087 cox_df = prepare_enhanced_cox_data(processed_df)
088
089 #拟合简化的 Cox 模型，专注特征选择
090 def fit_simplified_cox_model(cox_df):
091     # 特征工程
092     cox_df = cox_df.copy()
093     cox_df['BMI_centered'] = cox_df['BMI'] -
094 cox_df['BMI'].mean()
095     cox_df['BMI_centered_cubed'] =
096 cox_df['BMI_centered'] ** 3
097     cox_df['log_初始浓度'] = np.log(cox_df['初始浓度'] +
098 0.001)
099     cox_df['log_初始浓度_生产次数'] = cox_df['log_初始浓
100 度'] * cox_df['生产次数']
101
102     # 添加其他特征
103     if '体重' in cox_df.columns:
104         cox_df['体重_centered'] = cox_df['体重'] -
105 cox_df['体重'].mean()
106
107     if '误差 E' in cox_df.columns:
108         cox_df['误差 E_log'] = np.log(cox_df['误差
109 E'] + 0.001)
110
111     # 定义核心变量
112     core_vars = ['BMI_centered_cubed', '初始孕周', 'log_
113 初始浓度_生产次数']
114
115     # 添加可选变量
116     if '体重_centered' in cox_df.columns:
117         core_vars.append('体重_centered')
118

```

```

119     if '误差 E_log' in cox_df.columns:
120         core_vars.append('误差 E_log')
121
122     # 拟合模型
123     formula = " + ".join(core_vars)
124     print(f"使用公式: {formula}")
125
126     try:
127         model = CoxPHFitter()
128         model.fit(cox_df, duration_col='duration',
129 event_col='event', formula=formula)
130         print(f"模型 C-index:
131 {model.concordance_index_:.4f}")
132         print("\n 模型参数:")
133         print(model.summary)
134
135         # 变量重要性可视化
136         plt.figure(figsize=(12, 6))
137
138         # p 值条形图
139         plt.subplot(1, 2, 1)
140         summary = model.summary.sort_values('p')
141         summary['-log2(p)'] =
142 -np.log2(summary['p'])
143         summary['-log2(p)'].plot(kind='barh')
144         plt.title('变量显著性 (-log2(p)) ')
145         plt.xlabel('-log2(p)')
146
147         # 风险比条形图
148         plt.subplot(1, 2, 2)
149         summary =
150 model.summary.sort_values('exp(coef)', ascending=False)
151         summary['exp(coef)'].plot(kind='barh')
152         plt.title('风险比 (exp(coef)) ')
153         plt.xlabel('风险比')
154
155         plt.tight_layout()
156         plt.show()
157
158         return model, cox_df, formula
159
160     except Exception as e:
161         print(f"模型拟合失败: {str(e)}")
162         print("请更换基础模型")
163         return None, cox_df, ""
164
165 # 运行简化版 Cox 模型
166 best_cph, enhanced_cox_df, final_formula =
167 fit_simplified_cox_model(cox_df)

```

附录 7

Q3 求解 2.py

在最佳 formula 的结构上训练 Cox 模型，并进行 bmi-t_ideal 的预测
并进行误差分析和敏感性分析

001 """

```

002 在选择特征后探索出最佳 formula
003 在最佳 formula 的结构上训练 cox 模型，并进行 bmi-t_ideal 的预测
004 进行检测误差分析
005 最后交叉检验划分测试集验证灵敏度
006 """
007
008 import pandas as pd
009 import numpy as np
010 from lifelines import CoxPHFitter
011 import matplotlib.pyplot as plt
012 import os
013
014 from lifelines.utils import concordance_index
015
016 plt.rcParams['font.sans-serif'].insert(0, 'SimHei')
017 plt.rcParams['axes.unicode_minus'] = False
018 import matplotlib
019
020 matplotlib.use('TkAgg')
021
022 df = pd.read_excel(r"D:\pythonProject\数模准备\模型代码
023 \datas\processed_data1(2).xlsx")
024
025 #-----提前筛选用以检测误差的稳健数据-----
026 # 计算 25%分位数
027 q75 = df['E'].quantile(0.25)
028
029 df_sorted = df.sort_values(by='E')
030
031 processed_df = df_sorted[df_sorted['E'] <= q75]
032
033 def prepare_enhanced_cox_data(df):
034     """
035     准备增强的 Cox 模型数据，包含更多特征&特征
036     """
037     cox_data = df.copy()
038     cox_data['event'] = (cox_data['Y 染色体浓度'] >=
039 0.0395).astype(float)
040     result_data = []
041     for pregnant_id in cox_data['孕妇代码'].unique():
042         woman_data = cox_data[cox_data['孕妇代码'] ==
043 pregnant_id].sort_values('孕周')
044         initial_concentration = woman_data['Y 染色体浓度'].iloc[0]
045         initial_week = woman_data['孕周'].iloc[0]
046         reached_4_percent = woman_data[woman_data['Y 染色体浓度
047 ']] >= 0.0395]
048
049         record_base = {'孕妇代码': pregnant_id, '初始浓度':
050 initial_concentration, '初始孕周': initial_week}
051
052         if len(reached_4_percent) > 0:
053             first_reach = reached_4_percent.iloc[0]
054             record_base.update({'duration': first_reach['孕周
055 '], 'event': 1})
056             record_base.update(dict(first_reach))
057         else:
058             last_record = woman_data.iloc[-1]
059             record_base.update({'duration': last_record['孕周
060 '], 'event': 0})
061             record_base.update(dict(last_record))

```

```

062
063     result_data.append(record_base)
064     final_df = pd.DataFrame(result_data)
065     if '孕妇 BMI' in final_df.columns:
066         final_df.rename(columns={'孕妇 BMI': 'BMI'}, inplace=True)
067     return final_df
068
069
070 cox_df = prepare_enhanced_cox_data(processed_df)
071
072
073 def fit_new_formula_model(cox_df):
074
075     df = cox_df.copy()
076
077
078     df['BMI_centered'] = df['BMI'] - df['BMI'].mean()
079     df['BMI_centered_cubed'] = df['BMI_centered'] ** 3
080     df['log_初始浓度'] = np.log(df['初始浓度'].clip(lower=1e-6))
081     df['log_初始浓度_生产次数'] = df['log_初始浓度'] * df['生产次数']
082
083     formula = "BMI_centered_cubed + 初始孕周 + log_初始浓度_生产次数"
084
085     try:
086         cph = CoxPHFitter()
087         cph.fit(df, duration_col='duration', event_col='event',
088 formula=formula)
089         print(f"\n 模型公式: {formula}")
090         print(f"C-index: {cph.concordance_index_:.3f}")
091         print("\n 模型参数:");
092         print(cph.summary)
093         return cph, df
094     except Exception as e:
095         print(f"模型拟合失败: {str(e)}")
096         return None, df
097
098 best_cph, simplified_cox_df = fit_new_formula_model(cox_df)
099
100 #展示 Cox 模型的两个组成部分: 参数部分和非参数部分
101 def display_full_model_function(cph_model):
102     print("\n" + "=" * 20 + " Cox 模型的完整函数构成 " + "=" * 20)
103     #print("模型风险函数  $h(t|X) = h_0(t) * \exp(\beta_1 X_1 + \beta_2 X_2 + \dots)$ ")
104     # 构建公式字符串
105     formula_str = "exp("
106     for name, param_val in cph_model.params_.items():
107         formula_str += f"({param_val:.4f} * {name}) + "
108     formula_str = formula_str[:-3] + ")" # 去掉最后的 " + "
109     print("具体表达式为:", formula_str)
110
111
112 display_full_model_function(best_cph)
113
114 typical_initial_week = simplified_cox_df['初始孕周'].median()
115 typical_initial_concentration = simplified_cox_df['初始浓度'].median()
116 typical_production_count = simplified_cox_df['生产次数'].mean()
117
118 print("\n" + "=" * 20 + " 策略参数设定 " + "=" * 20)
119 print(f"典型初始孕周: {typical_initial_week:.2f}")
120 print(f"典型初始浓度: {typical_initial_concentration:.4f}")
121 print(f"典型生产次数: {typical_production_count}")

```

```

122 print("=" * 55)
123
124 def solve_for_ideal_time(bmi_value, cph_model, df_featured,
125                          typical_week,
126                          typical_conc, typical_prod_count):
127     if cph_model is None: return np.nan
128
129     bmi_mean = df_featured['BMI'].mean()
130
131     X_pred = pd.DataFrame({
132         'BMI_centered_cubed': [(bmi_value - bmi_mean) ** 3],
133         '初始孕周': [typical_week],
134         'log_初始浓度_生产次数': [np.log(max(typical_conc, 0) +
135 1e-6) * typical_prod_count]
136     })
137
138     time_95 = cph_model.predict_percentile(X_pred, p=0.05)
139     return time_95
140
141 bmi_range = np.arange(simplified_cox_df['BMI'].min(),
142                      simplified_cox_df['BMI'].max(), 0.1)
143 ideal_times = [solve_for_ideal_time(
144     b,
145     best_cph,
146     simplified_cox_df,
147     typical_initial_week,
148     typical_initial_concentration,
149     typical_production_count
150 ) for b in bmi_range]
151
152 bmi_time_df = pd.DataFrame({'BMI': bmi_range, 'T_ideal': ideal_times})
153
154 #print("\n 成功生成 (BMI, T_ideal) 数据集，部分数据如下：")
155 #print(bmi_time_df.head())
156 #print("-" * 50)
157
158 save_dir = r"/result"
159 os.makedirs(save_dir, exist_ok=True)
160 file_path = os.path.join(save_dir, "bmi_time_q3_new_formula 低.xlsx")
161 bmi_time_df.to_excel(file_path, index=False)
162 #print(f"数据集已成功保存至: {file_path}")
163
164 # -----灵敏度-----
165
166
167 def sensitivity_analysis_new_model(cox_df, test_size=0.2,
168 random_state=42):
169
170     from sklearn.model_selection import train_test_split
171
172     train_df, test_df = train_test_split(
173         cox_df,
174         test_size=test_size,
175         random_state=random_state,
176         stratify=cox_df['event'] # 保持事件比例一致
177     )
178
179     print(f"训练集大小: {len(train_df)}")
180     print(f"测试集大小: {len(test_df)}")
181     print(f"训练集中事件比例: {train_df['event'].mean():.3f}")

```



```

182     print(f"测试集中事件比例: {test_df['event'].mean():.3f}")
183
184
185     train_df = train_df.copy()
186     train_df['BMI_centered'] = train_df['BMI'] -
187 train_df['BMI'].mean()
188     train_df['BMI_centered_cubed'] = train_df['BMI_centered'] ** 3
189     train_df['log_初始浓度'] = np.log(train_df['初始浓度
190 '].clip(lower=1e-6))
191     train_df['log_初始浓度_生产次数'] = train_df['log_初始浓度'] *
192 train_df['生产次数']
193
194     formula = "BMI_centered_cubed + 初始孕周 + log_初始浓度_生产次数"
195
196     cph_train = CoxPHFitter()
197     cph_train.fit(train_df, duration_col='duration',
198 event_col='event', formula=formula)
199
200     print("\n训练集模型性能:")
201     print(f"C-index: {cph_train.concordance_index:.3f}")
202     print(cph_train.summary)
203
204
205     test_df = test_df.copy()
206     bmi_mean_train = train_df['BMI'].mean()
207     test_df['BMI_centered'] = test_df['BMI'] - bmi_mean_train
208     test_df['BMI_centered_cubed'] = test_df['BMI_centered'] ** 3
209     test_df['log_初始浓度'] = np.log(test_df['初始浓度
210 '].clip(lower=1e-6))
211     test_df['log_初始浓度_生产次数'] = test_df['log_初始浓度'] *
212 test_df['生产次数']
213
214     # 在测试集上评估模型
215     try:
216         # 预测风险分数
217         test_df['predicted_risk'] =
218 cph_train.predict_partial_hazard(test_df)
219
220         # 计算C-index
221         test_c_index = concordance_index(
222             test_df['duration'],
223             -test_df['predicted_risk'],
224             test_df['event']
225         )
226         # print(f"\n 测试集C-index: {test_c_index:.3f}")
227
228         # 计算训练集和测试集的C-index 差异
229         c_index_diff = abs(cph_train.concordance_index_ -
230 test_c_index)
231         #print(f"C-index 差异: {c_index_diff:.3f}")
232
233     except Exception as e:
234         print(f"测试集评估出错: {e}")
235
236     return cph_train, train_df, test_df
237
238 # 执行灵敏性分析
239 cph_trained, train_data, test_data =
240 sensitivity_analysis_new_model(simplified_cox_df)
241

```

```

242
243 # 交叉验证评估模型稳定性
244 def cross_validate_cox_new_model(cox_df, n_splits=5, random_state=42):
245
246     from sklearn.model_selection import KFold
247
248     cox_df_processed = cox_df.copy()
249     cox_df_processed['BMI_centered'] = cox_df_processed['BMI'] -
250 cox_df_processed['BMI'].mean()
251     cox_df_processed['BMI_centered_cubed'] =
252 cox_df_processed['BMI_centered'] ** 3
253     cox_df_processed['log_初始浓度'] = np.log(cox_df_processed['初始
254 浓度']).clip(lower=1e-6))
255     cox_df_processed['log_初始浓度_生产次数'] = cox_df_processed['log_
256 初始浓度'] * cox_df_processed['生产次数']
257
258     kf = KFold(n_splits=n_splits, shuffle=True,
259 random_state=random_state)
260     c_indices = []
261
262     for train_index, test_index in kf.split(cox_df_processed):
263         train_fold = cox_df_processed.iloc[train_index]
264         test_fold = cox_df_processed.iloc[test_index]
265
266         # 训练模型
267         cph_fold = CoxPHFitter()
268         cph_fold.fit(train_fold, duration_col='duration',
269 event_col='event',
270                     formula="BMI_centered_cubed + 初
271 始孕周 + log_初始浓度_生产次数")
272
273         test_fold = test_fold.copy()
274         test_fold['predicted_risk'] =
275 cph_fold.predict_partial_hazard(test_fold)
276
277         c_index = concordance_index(
278             test_fold['duration'],
279             -test_fold['predicted_risk'],
280             test_fold['event']
281         )
282         c_indices.append(c_index)
283
284     # 输出交叉验证结果
285     #print(f"\n{'-' * 50}")
286     #print(f"{n_splits}折交叉验证结果:")
287     #print(f"C-index 平均值: {np.mean(c_indices):.3f}")
288     #print(f"C-index 标准差: {np.std(c_indices):.3f}")
289     #print(f"C-index 范围: [{min(c_indices):.3f},
290 {max(c_indices):.3f}])")
291
292     return c_indices
293
294 cv_results = cross_validate_cox_new_model(simplified_cox_df)
295
296
297 # 评估模型在不同 BMI 分组上的表现
298 def evaluate_by_bmi_groups_new_model(model, test_df, n_groups=3):
299
300     # 创建 BMI 分组
301     test_df = test_df.copy()

```

```

302     test_df['BMI_group'] = pd.qcut(test_df['BMI'], n_groups,
303 labels=[f'Group {i + 1}' for i in range(n_groups)])
304
305     group_c_indices = {}
306
307     for group in test_df['BMI_group'].unique():
308         group_data = test_df[test_df['BMI_group'] == group].copy()
309
310         # 预测风险分数
311         group_data['predicted_risk'] =
312 model.predict_partial_hazard(group_data)
313
314         c_index = concordance_index(
315             group_data['duration'],
316             -group_data['predicted_risk'],
317             group_data['event']
318         )
319         group_c_indices[group] = c_index
320
321
322     return group_c_indices
323
324
325 # 执行分组评估
326 bmi_group_results = evaluate_by_bmi_groups_new_model(cph_trained,
327 test_data)
328
329
330 # 评估模型在不同生产次数分组上的表现
331 def evaluate_by_production_groups(model, test_df):
332     """
333     按生产次数分组评估新模型性能
334     """
335     # 创建生产次数分组
336     test_df = test_df.copy()
337
338     group_c_indices = {}
339
340     for group in test_df['production_group'].unique():
341         group_data = test_df[test_df['production_group'] ==
342 group].copy()
343
344         # 预测风险分数
345         group_data['predicted_risk'] =
346 model.predict_partial_hazard(group_data)
347
348         # 计算该组的 C-index
349         c_index = concordance_index(
350             group_data['duration'],
351             -group_data['predicted_risk'],
352             group_data['event']
353         )
354         group_c_indices[group] = c_index
355
356     # 输出结果
357     print(f"\n 按生产次数分组评估结果:")
358     for group, c_index in group_c_indices.items():
359         print(f"{group}: C-index = {c_index:.3f}")
360
361     return group_c_indices

```

```

production_group_results = evaluate_by_production_groups(cph_trained,
test_data)

#print("\n 敏感性分析完成!")

```

附录 8

Q3 约束动态规划.py

问题 3 新增约束的动态规划

```

001 '''
002 问题 3 新增约束的动态规划
003 '''
004 import numpy as np
005 import pandas as pd
006 import matplotlib.pyplot as plt
007 import warnings
008
009 warnings.filterwarnings('ignore')
010 import matplotlib
011
012 matplotlib.use('TkAgg')
013 plt.rcParams['font.sans-serif'].insert(0, 'SimHei')
014 plt.rcParams['axes.unicode_minus'] = False # 解决负号显示
015 问题
016
017 # 读取数据并确保按 BMI 排序
018 df = pd.read_excel(r"D:\pythonProject\数模准备\模型代码
019 \result\bmi_time1.xlsx"
020
021 )
022 df_sorted =
023 df.sort_values(by='BMI').reset_index(drop=True)
024
025 # 定义全局硬约束
026 MIN_PASS_RATE = 0.8
027 MIN_GROUP_SIZE = 5
028
029 def compute_group_properties(t_ideals):
030     if len(t_ideals) == 0: return 0, 0, 1.0
031     sorted_t = np.array(t_ideals);
032     n = len(sorted_t)
033     prefix_sum = np.cumsum(sorted_t);
034     prefix_sum_sq = np.cumsum(sorted_t ** 2)
035     total_sum = prefix_sum[-1];
036     total_sum_sq = prefix_sum_sq[-1]
037     min_risk = float('inf');
038     best_I = np.mean(sorted_t)
039     indices = np.arange(n);
040     n1 = indices + 1;
041     n2 = n - n1
042     S1 = prefix_sum;
043     S2 = total_sum - S1;
044     sum_sq1 = prefix_sum_sq;
045     sum_sq2 = total_sum_sq - sum_sq1
046     risks = 0.27 * (n1 * sorted_t ** 2 - 2 * sorted_t *
047 S1 + sum_sq1) + \
048 0.83 * (n2 * sorted_t ** 2 - 2 *
049 sorted_t * S2 + sum_sq2)

```

```

050     min_risk_idx = np.argmin(risks);
051     min_risk = risks[min_risk_idx];
052     best_T = sorted_t[min_risk_idx]
053     i = np.arange(n - 1);
054     n1_interval = i + 1;
055     n2_interval = n - n1_interval
056     S1_interval = prefix_sum[i];
057     S2_interval = total_sum - S1_interval
058     denominator = 0.27 * n1_interval + 0.83 * n2_interval
059     T0_candidates = (0.27 * S1_interval + 0.83 *
060 S2_interval) / denominator
061     valid_mask = (T0_candidates >= sorted_t[i]) &
062 (T0_candidates < sorted_t[i + 1])
063     if np.any(valid_mask):
064         valid_i = i[valid_mask];
065         valid_T0 = T0_candidates[valid_mask]
066         risk_T0 = 0.27 * (
067
068 n1_interval[valid_mask] * valid_T0 ** 2 - 2 * valid_T0 *
069 S1_interval[valid_mask] + prefix_sum_sq[
070         valid_i]) + \
071         0.83 *
072 (n2_interval[valid_mask] * valid_T0 ** 2 - 2 * valid_T0 *
073 S2_interval[valid_mask] + (
074         total_sum_sq -
075 prefix_sum_sq[valid_i]))
076         min_risk_T0_idx = np.argmin(risk_T0)
077         if risk_T0[min_risk_T0_idx] < min_risk:
078             min_risk = risk_T0[min_risk_T0_idx];
079             best_T = valid_T0[min_risk_T0_idx]
080     pass_count = np.sum(sorted_t <= best_T);
081     pass_rate = pass_count / n
082     return min_risk, best_T, pass_rate
083
084
085 # --- 动态规划主函数 (加入最小分组约束) ---
086 def optimal_partitioning(BMI, T_ideal, lambda_val):
087     N = len(BMI)
088     if N == 0: return [], [], 0, 0, 0
089
090     dp = np.full(N + 1, np.inf)
091     dp[0] = 0
092     split_points = np.zeros(N + 1, dtype=int)
093
094     props_cache = {}
095     for j in range(N):
096         for i in range(j + 1, N + 1):
097             props_cache[(j, i)] =
098 compute_group_properties(T_ideal[j:i])
099
100     for i in range(1, N + 1):
101         for j in range(0, i):
102             # 如果当前尝试的组 [j, i-1] 太小, 则跳
103 过
104             if (i - j) < MIN_GROUP_SIZE:
105                 continue
106
107             risk_val, T_opt, pass_rate =
108 props_cache[(j, i)]
109

```

```

110         if pass_rate < MIN_PASS_RATE:
111             cost_val = float('inf')
112         else:
113             cost_val = risk_val +
114 lambda_val * T_opt
115
116         if dp[j] + cost_val < dp[i]:
117             dp[i] = dp[j] + cost_val
118             split_points[i] = j
119
120     # (回溯部分不变)
121     groups = [];
122     current = N
123     while current > 0:
124         prev = split_points[current]
125         if prev == current: # 无法找到有效分割点
126             print("警告: 无法为所有数据点找到有效
127 的分组方案。可能需要放宽约束。")
128             return [], [], 0, 0, float('inf')
129         groups.append((prev, current - 1))
130         current = prev
131     groups.reverse()
132
133     group_Ts = [];
134     total_risk = 0;
135     total_T = 0
136     for (start, end) in groups:
137         risk, T_opt, _ = props_cache[(start, end + 1)]
138         group_Ts.append(T_opt)
139         total_risk += risk
140         total_T += T_opt
141
142     return groups, group_Ts, total_risk, total_T, dp[N]
143
144 #参数调优
145 def tune_lambda(BMI, T_ideal, lambda_values=None):
146     if lambda_values is None:
147         lambda_values = np.logspace(-3, 3, 40)
148
149     results = []
150     for lambda_val in lambda_values:
151         groups, group_Ts, total_risk, total_T,
152 total_cost = optimal_partitioning(
153             BMI, T_ideal, lambda_val)
154         num_groups = len(groups)
155         avg_T = total_T / num_groups if num_groups >
156 0 else 0
157         results.append({
158             'lambda': lambda_val, 'total_risk':
159 total_risk, 'avg_T': avg_T,
160             'total_cost': total_cost,
161             'num_groups': num_groups
162         })
163     results_df = pd.DataFrame(results)
164
165     results_df_valid =
166 results_df[results_df['num_groups'] > 0].copy()
167     if len(results_df_valid) == 0:
168         print("警告: 在所有测试的λ下都无法找到满足约束
169 的分组!")

```

```

170         return pd.DataFrame(), None
171
172
173     # 调整综合得分的权重, 加大对组数的惩罚
174     for col in ['total_risk', 'avg_T', 'num_groups']:
175         min_val, max_val =
176 results_df_valid[col].min(), results_df_valid[col].max()
177         if (max_val - min_val) > 1e-6:
178             results_df_valid[f'norm_{col}'] =
179 (results_df_valid[col] - min_val) / (max_val - min_val)
180         else:
181             results_df_valid[f'norm_{col}'] = 0
182
183     # 权重: 风险(30%), 平均 T(10%), 组数(60%)
184     results_df_valid['score'] = 0.3 *
185 results_df_valid['norm_total_risk'] + \
186
187 0.1 * results_df_valid['norm_avg_T'] + \
188
189 0.6 * results_df_valid['norm_num_groups']
190
191     best_idx = results_df_valid['score'].idxmin()
192     best_lambda = results_df_valid.loc[best_idx,
193 'lambda']
194
195     return results_df, best_lambda
196
197 #分组结果可视化
198 def plot_partitioning(BMI, T_ideal, groups, group_Ts,
199 title="BMI 分组结果"):
200     plt.figure(figsize=(12, 8));
201     colors = plt.cm.get_cmap('viridis', len(groups))
202     for i, (start, end) in enumerate(groups):
203         group_bmi = BMI[start:end + 1];
204         group_t_ideal = T_ideal[start:end + 1];
205         best_T = group_Ts[i]
206         plt.scatter(group_bmi, group_t_ideal,
207 color=colors(i), label=f'组 {i + 1}', s=50, alpha=0.7)
208         plt.hlines(y=best_T, xmin=group_bmi.min(),
209 xmax=group_bmi.max(), color=colors(i), linestyle='--')
210         pass_count = np.sum(group_t_ideal <=
211 best_T);
212         pass_rate = pass_count / len(group_t_ideal)
213         plt.text(np.mean(group_bmi), best_T + 0.1,
214 f'T={best_T:.2f}\n 达标率={pass_rate:.1%}',
215 color=colors(i), fontsize=9,
216 ha='center', va='bottom',
217 bbox=dict(boxstyle='round,pad=0.2', fc='white',
218 ec='none', alpha=0.8))
219         for i in range(1, len(groups)):
220             boundary_bmi = (BMI[groups[i][0] - 1] +
221 BMI[groups[i][0]]) / 2
222             plt.axvline(x=boundary_bmi, color='gray',
223 linestyle=':', alpha=0.5)
224     plt.xlabel('BMI');
225     plt.ylabel('T_ideal');
226     plt.title(title);
227     plt.legend();
228     plt.grid(True, alpha=0.3);
229     plt.tight_layout();

```

```

plt.show()

if __name__ == "__main__":
    BMI_values = df_sorted['BMI'].values
    T_ideal_values = df_sorted['T_ideal'].values

    #print(f"开始调优, 约束: 每组达标率 >=
{MIN_PASS_RATE:.0%}, 每组最少样本数 >= {MIN_GROUP_SIZE}")
    results_df, best_lambda = tune_lambda(BMI_values,
T_ideal_values)

    if best_lambda is not None:
        print(f"\n 使用最佳λ值({best_lambda:.4f})进
行最终分组...")
        groups, group_Ts, total_risk, total_T,
total_cost = optimal_partitioning(
            BMI_values, T_ideal_values,
best_lambda)

        if groups:
            for i, (start, end) in
enumerate(groups):
                _, _, pass_rate =
compute_group_properties(T_ideal_values[start:end + 1])
                # print(f"组 {i + 1}: BMI 范围
[{BMI_values[start]:.2f}, {BMI_values[end]:.2f}], "
                #f"样本数 {end -
start + 1}, 最佳 T={group_Ts[i]:.2f}, 达标率
={pass_rate:.1%}")

                num_groups = len(groups)
                avg_T = total_T / num_groups if
num_groups > 0 else 0

                plot_partitioning(BMI_values,
T_ideal_values, groups, group_Ts,

title=f"最终BMI分组结果 (λ={best_lambda:.4f}, {num_groups}
个分组)")
            else:
                print("未能找到满足所有约束条件的分组
方案。")

```

附录 9

Q4_logist.py

逻辑回归检测异常值

```

001 import pandas as pd
002 import numpy as np
003 import matplotlib.pyplot as plt
004 from sklearn.linear_model import LogisticRegression
005 from sklearn.preprocessing import StandardScaler,
006 label_binarize
007 from sklearn.metrics import (classification_report,
008 roc_curve, auc,
009
010 roc_auc_score, precision_recall_curve,
011

```



```

012 average_precision_score, confusion_matrix)
013 from sklearn.model_selection import (cross_val_score,
014 StratifiedKFold,
015
016 train_test_split)
017 from sklearn.multiclass import OneVsRestClassifier
018 from sklearn.exceptions import ConvergenceWarning
019 import seaborn as sns
020 import warnings
021
022 warnings.filterwarnings('ignore',
023 category=ConvergenceWarning)
024 plt.rcParams['font.sans-serif'].insert(0, 'SimHei')
025 plt.rcParams['axes.unicode_minus'] = False
026 import matplotlib
027
028 matplotlib.use('TkAgg')
029
030 def process_l1_logistic_regression(df):
031     columns_to_drop = ['非整倍体数量', '18 异常', '21 异
032 常', '13 异常']
033     X = df.drop(columns_to_drop, axis=1)
034     y = df['非整倍体数量']
035
036     X_train, X_test, y_train, y_test = train_test_split(
037         X, y, test_size=0.3, stratify=y,
038         random_state=42
039     )
040
041     scaler = StandardScaler()
042     X_train_scaled = scaler.fit_transform(X_train)
043     X_test_scaled = scaler.transform(X_test)
044
045     base_estimator = LogisticRegression(
046         penalty='l1',
047         solver='liblinear',
048         C=0.1,
049         class_weight='balanced',
050         max_iter=1000,
051         random_state=42
052     )
053     model = OneVsRestClassifier(base_estimator)
054
055     cv = StratifiedKFold(n_splits=5, shuffle=True,
056 random_state=42)
057     cv_accuracy = cross_val_score(model,
058 X_train_scaled, y_train, cv=cv, scoring='accuracy')
059     cv_auc = cross_val_score(model, X_train_scaled,
060 y_train, cv=cv, scoring='roc_auc_ovr')
061     cv_f1 = cross_val_score(model, X_train_scaled,
062 y_train, cv=cv, scoring='f1_macro')
063
064     model.fit(X_train_scaled, y_train)
065
066     y_train_pred = model.predict(X_train_scaled)
067     train_report = classification_report(y_train,
068 y_train_pred, output_dict=False)
069
070     y_test_pred = model.predict(X_test_scaled)
071     test_report = classification_report(y_test,

```

```

072 y_test_pred, output_dict=False)
073
074 cm_test = confusion_matrix(y_test, y_test_pred)
075
076 # 获取特征重要性
077 coef_list = [estimator.coef_.ravel() for estimator
078 in model.estimators_]
079 coef_matrix = np.array(coef_list)
080 importance = np.max(np.abs(coef_matrix), axis=0)
081
082 # 创建特征重要性 DataFrame
083 feature_importance = pd.DataFrame({
084     '特征': X.columns,
085     '重要性': importance
086 })
087
088 # 获取 top5 特征
089 top5_features = feature_importance.sort_values('重
090 要性', ascending=False).head(5)
091
092 # 计算训练集 ROC 曲线和 AUC
093 y_train_bin = label_binarize(y_train,
094 classes=np.unique(y))
095 n_classes = y_train_bin.shape[1]
096
097 # 获取训练集预测概率
098 y_train_score =
099 model.predict_proba(X_train_scaled)
100
101 # 计算每个类别的 ROC 曲线和 AUC
102 fpr_train = dict()
103 tpr_train = dict()
104 roc_auc_train = dict()
105
106 for i in range(n_classes):
107     fpr_train[i], tpr_train[i], _ =
108     roc_curve(y_train_bin[:, i], y_train_score[:, i])
109     roc_auc_train[i] = auc(fpr_train[i],
110 tpr_train[i])
111
112     all_fpr_train =
113     np.unique(np.concatenate([fpr_train[i] for i in
114 range(n_classes)]))
115     mean_tpr_train = np.zeros_like(all_fpr_train)
116     for i in range(n_classes):
117         mean_tpr_train += np.interp(all_fpr_train,
118 fpr_train[i], tpr_train[i])
119     mean_tpr_train /= n_classes
120
121     fpr_train["macro"] = all_fpr_train
122     tpr_train["macro"] = mean_tpr_train
123     roc_auc_train["macro"] = auc(fpr_train["macro"],
124 tpr_train["macro"])
125
126 # 准备训练集 ROC/AUC 数据
127 roc_data_train = {
128     'fpr': fpr_train,
129     'tpr': tpr_train,
130     'roc_auc': roc_auc_train,
131     'n_classes': n_classes

```

```

132     }
133
134     # 计算训练集 PR 曲线和 AP
135     precision_train = dict()
136     recall_train = dict()
137     average_precision_train = dict()
138
139     for i in range(n_classes):
140         precision_train[i], recall_train[i], _ =
141 precision_recall_curve(y_train_bin[:, i],
142 y_train_score[:, i])
143         average_precision_train[i] =
144 average_precision_score(y_train_bin[:, i],
145 y_train_score[:, i])
146
147     # 计算宏平均 PR 曲线和 AP
148     all_recall_train =
149 np.unique(np.concatenate([recall_train[i] for i in
150 range(n_classes)]))
151     mean_precision_train =
152 np.zeros_like(all_recall_train)
153     for i in range(n_classes):
154         mean_precision_train +=
155 np.interp(all_recall_train, recall_train[i][::-1],
156 precision_train[i][::-1])
157     mean_precision_train /= n_classes
158
159     precision_train["macro"] = mean_precision_train
160     recall_train["macro"] = all_recall_train
161     average_precision_train["macro"] =
162 np.mean(list(average_precision_train.values()))
163
164     # 准备训练集 PR/AP 数据
165     pr_data_train = {
166         'precision': precision_train,
167         'recall': recall_train,
168         'average_precision':
169 average_precision_train,
170         'n_classes': n_classes
171     }
172
173     # 计算测试集 ROC 曲线和 AUC
174     y_test_bin = label_binarize(y_test,
175 classes=np.unique(y))
176     y_test_score = model.predict_proba(X_test_scaled)
177
178     # 计算每个类别的 ROC 曲线和 AUC
179     fpr_test = dict()
180     tpr_test = dict()
181     roc_auc_test = dict()
182
183     for i in range(n_classes):
184         fpr_test[i], tpr_test[i], _ =
185 roc_curve(y_test_bin[:, i], y_test_score[:, i])
186         roc_auc_test[i] = auc(fpr_test[i],
187 tpr_test[i])
188
189     # 计算宏平均 ROC 曲线和 AUC
190     all_fpr_test =
191 np.unique(np.concatenate([fpr_test[i] for i in

```

```

192     range(n_classes]))
193     mean_tpr_test = np.zeros_like(all_fpr_test)
194     for i in range(n_classes):
195         mean_tpr_test += np.interp(all_fpr_test,
196 fpr_test[i], tpr_test[i])
197     mean_tpr_test /= n_classes
198
199     fpr_test["macro"] = all_fpr_test
200     tpr_test["macro"] = mean_tpr_test
201     roc_auc_test["macro"] = auc(fpr_test["macro"],
202 tpr_test["macro"])
203
204     # 准备测试集 ROC/AUC 数据
205     roc_data_test = {
206         'fpr': fpr_test,
207         'tpr': tpr_test,
208         'roc_auc': roc_auc_test,
209         'n_classes': n_classes
210     }
211
212     # 计算测试集 PR 曲线和 AP
213     precision_test = dict()
214     recall_test = dict()
215     average_precision_test = dict()
216
217     for i in range(n_classes):
218         precision_test[i], recall_test[i], _ =
219 precision_recall_curve(y_test_bin[:, i], y_test_score[:,
220 i])
221         average_precision_test[i] =
222 average_precision_score(y_test_bin[:, i], y_test_score[:,
223 i])
224
225     # 计算宏平均 PR 曲线和 AP
226     all_recall_test =
227 np.unique(np.concatenate([recall_test[i] for i in
228 range(n_classes)]))
229     mean_precision_test =
230 np.zeros_like(all_recall_test)
231     for i in range(n_classes):
232         mean_precision_test +=
233 np.interp(all_recall_test, recall_test[i][::-1],
234 precision_test[i][::-1])
235     mean_precision_test /= n_classes
236
237     precision_test["macro"] = mean_precision_test
238     recall_test["macro"] = all_recall_test
239     average_precision_test["macro"] =
240 np.mean(list(average_precision_test.values()))
241
242     # 准备测试集 PR/AP 数据
243     pr_data_test = {
244         'precision': precision_test,
245         'recall': recall_test,
246         'average_precision':
247 average_precision_test,
248         'n_classes': n_classes
249     }
250
251     # 准备交叉验证结果

```

```

252     cv_result = {
253         'accuracy': {
254             'scores': cv_accuracy,
255             'mean': np.mean(cv_accuracy),
256             'std': np.std(cv_accuracy)
257         },
258         'auc': {
259             'scores': cv_auc,
260             'mean': np.mean(cv_auc),
261             'std': np.std(cv_auc)
262         },
263         'f1': {
264             'scores': cv_f1,
265             'mean': np.mean(cv_f1),
266             'std': np.std(cv_f1)
267         }
268     }
269
270     return (model, train_report, test_report,
271 top5_features, cv_result,
272         roc_data_train, pr_data_train,
273 roc_data_test, pr_data_test, cm_test)
274
275
276 def plot_roc_curves(roc_data, title, dataset_type="训练集
277 "):
278     """绘制 ROC 曲线"""
279     plt.figure(figsize=(10, 8))
280
281     # 绘制每个类别的 ROC 曲线
282     for i in range(roc_data['n_classes']):
283         plt.plot(roc_data['fpr'][i],
284 roc_data['tpr'][i],
285                 label=f'类别 {i} (AUC =
286 {roc_data["roc_auc"][i]:.2f})')
287
288     # 绘制宏平均 ROC 曲线
289     plt.plot(roc_data['fpr']['macro'],
290 roc_data['tpr']['macro'],
291             label=f'宏平均 (AUC =
292 {roc_data["roc_auc"]["macro"]:.2f})',
293             linestyle='--', linewidth=2)
294
295     # 绘制对角线
296     plt.plot([0, 1], [0, 1], 'k--', lw=2)
297
298     plt.xlim([0.0, 1.0])
299     plt.ylim([0.0, 1.05])
300     plt.xlabel('假阳性率 (False Positive Rate)',
301 fontsize=12)
302     plt.ylabel('真阳性率 (True Positive Rate)',
303 fontsize=12)
304     plt.title(f'{title} - {dataset_type} ROC 曲线',
305 fontsize=14)
306     plt.legend(loc="lower right", fontsize=10)
307     plt.grid(True, alpha=0.3)
308     plt.show()
309
310
311 def plot_pr_curves(pr_data, title, dataset_type="训练集"):

```

```

312     """绘制 PR 曲线"""
313     plt.figure(figsize=(10, 8))
314
315     # 绘制每个类别的 PR 曲线
316     for i in range(pr_data['n_classes']):
317         plt.plot(pr_data['recall'][i],
318                  pr_data['precision'][i],
319                  label=f'类别 {i} (AP =
320                  {pr_data["average_precision"][i]:.2f})')
321
322     # 绘制宏平均 PR 曲线
323     plt.plot(pr_data['recall']['macro'],
324              pr_data['precision']['macro'],
325              label=f'宏平均 (AP =
326              {pr_data["average_precision"]["macro"]:.2f})',
327              linestyle='--', linewidth=2)
328
329     plt.xlim([0.0, 1.0])
330     plt.ylim([0.0, 1.05])
331     plt.xlabel('召回率 (Recall)', fontsize=12)
332     plt.ylabel('精确率 (Precision)', fontsize=12)
333     plt.title(f'{title} - {dataset_type} PR 曲线',
334              fontsize=14)
335     plt.legend(loc="upper right", fontsize=10)
336     plt.grid(True, alpha=0.3)
337     plt.show()
338
339
340 def plot_confusion_matrices(confusion_matrices, titles):
341     """绘制四个数据集的混淆矩阵"""
342     plt.figure(figsize=(16, 12))
343
344     cmap = sns.diverging_palette(220, 10, as_cmap=True)
345
346     for i, (cm, title) in
347 enumerate(zip(confusion_matrices, titles)):
348         plt.subplot(2, 2, i + 1)
349
350         cm_normalized = cm.astype('float') /
351 cm.sum(axis=1)[:, np.newaxis]
352
353         annotations =
354 np.array([f"{count}\n({percent:.1%})"
355
356 for count, percent in zip(cm.flatten(),
357 cm_normalized.flatten())])
358         annotations = annotations.reshape(cm.shape)
359
360         # 绘制热力图
361         sns.heatmap(cm_normalized,
362                    annot=annotations, fmt='', cmap=cmap,
363                    cbar=i == 0, # 只在
364                    第一个子图显示颜色条
365                    vmin=0, vmax=1, #
366                    统一颜色范围
367                    annot_kws={"size":
368                    10})
369
370         plt.title(f'{title} - 混淆矩阵',
371                  fontsize=12)

```

```

372         plt.xlabel('预测类别', fontsize=10)
373         plt.ylabel('真实类别', fontsize=10)
374
375         # 设置刻度标签
376         plt.xticks([0.5, 1.5, 2.5], ['0', '1', '2'])
        plt.yticks([0.5, 1.5, 2.5], ['0', '1', '2'])

        plt.tight_layout()
        plt.show()

# 加载数据
df1 = pd.read_excel(r"D:\pythonProject\数模准备\模型代码\
\datas\df_fixed.xlsx")
df2 = pd.read_excel(r"D:\pythonProject\数模准备\模型代码\
\datas\df_over.xlsx")
df3 = pd.read_excel(r"D:\pythonProject\数模准备\模型代码\
\datas\df_under.xlsx")
df4 = pd.read_excel(r"D:\pythonProject\数模准备\模型代码\
\datas\df_smote.xlsx")

# 处理所有 DataFrame
results = {}
dataframes = {'df1': df1, 'df2': df2, 'df3': df3, 'df4': df4}

# 收集所有数据集的混淆矩阵
confusion_matrices = []
titles = []

for name, df in dataframes.items():
    print(f"\n{'=' * 50}\n处理 {name}...\n{'=' * 50}")

    # 处理数据并获取结果
    (model, train_report, test_report, top5, cv_result,
     roc_data_train, pr_data_train, roc_data_test,
     pr_data_test, cm_test) =
    process_l1_logistic_regression(df)

    # 存储结果
    results[name] = {
        'model': model,
        'train_report': train_report,
        'test_report': test_report,
        'top5_features': top5,
        'cv_result': cv_result,
        'roc_data_train': roc_data_train,
        'pr_data_train': pr_data_train,
        'roc_data_test': roc_data_test,
        'pr_data_test': pr_data_test,
        'cm_test': cm_test
    }

    # 收集混淆矩阵和标题
    confusion_matrices.append(cm_test)
    titles.append(name)

    # 打印 Top5 特征
    #print(f"\n{name} 的 Top5 重要特征:\n{'-' * 30}")
    #print(top5.to_string(index=False))

```

```

# 绘制训练集 ROC 曲线
plot_roc_curves(roc_data_train, name, "训练集")

# 绘制训练集 PR 曲线
plot_pr_curves(pr_data_train, name, "训练集")

# 绘制测试集 ROC 曲线
plot_roc_curves(roc_data_test, name, "测试集")

# 绘制测试集 PR 曲线
plot_pr_curves(pr_data_test, name, "测试集")

# 绘制所有数据集的混淆矩阵
print(f"\n{'=' * 50}\n所有数据集混淆矩阵\n{'=' * 50}")
plot_confusion_matrices(confusion_matrices, titles)

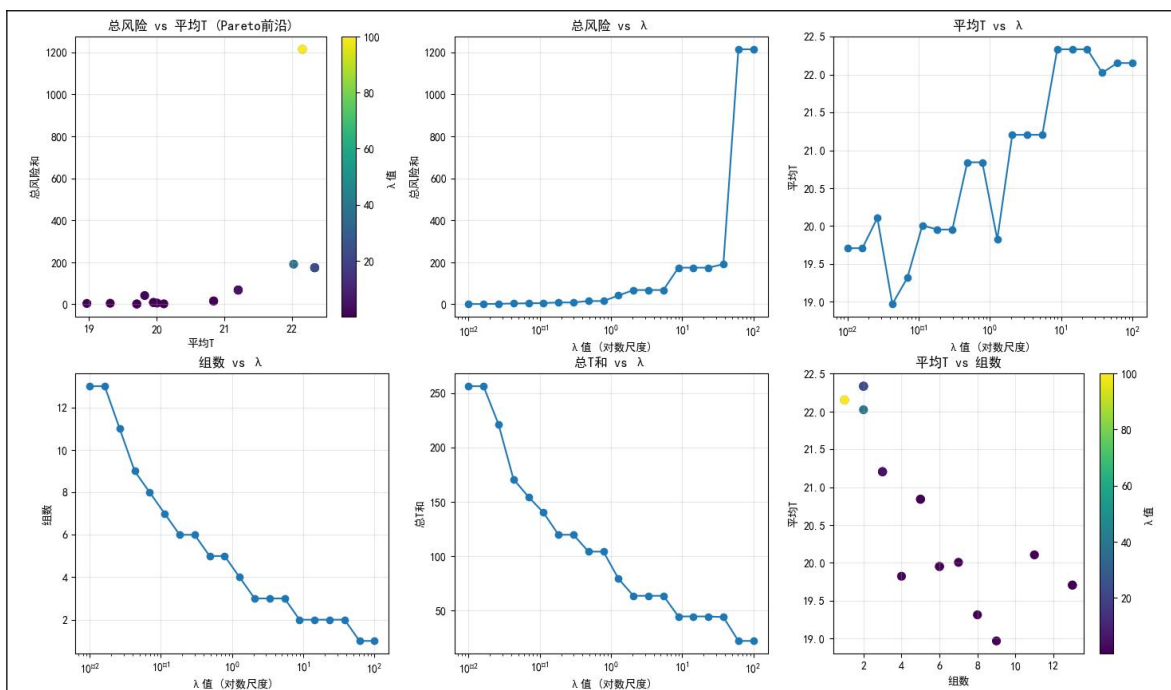
# 汇总所有数据集的测试集结果
summary_data = []
for name, result in results.items():
    summary_data.append({
        '数据集': name,
        '交叉验证准确率':
result['cv_result']['accuracy']['mean'],
        '交叉验证 AUC':
result['cv_result']['auc']['mean'],
        '交叉验证 F1':
result['cv_result']['f1']['mean'],
        '测试集 AUC':
result['roc_data_test']['roc_auc']['macro'],
        '测试集 AP':
result['pr_data_test']['average_precision']['macro']
    })

summary_df = pd.DataFrame(summary_data)
print(summary_df.to_string(index=False))

```

附录 11

Pareto 调参可视化



附录 12

逻辑回归四组数据集交叉检验 ROC 和 PR 图

